

Contents

1. Introduction	6
1.1. Project Background.....	6
1.2. Project Introduction.....	6
1.3. Problem Statement.....	7
1.4. Aim of Project	7
1.5. Research Question.....	7
1.6. Project Objectives.....	8
1.7. Scope of Research	8
2. Literature Review	8
2.1 Automated Text Summarization	9
2.2 Deep Learning in Natural Language Processing	9
2.3 Tools and Technologies for Text Extraction and Summarization	10
2.4 Existing Methodologies in Paper Analysis	11
2.5 Challenges and Limitations in Automated Summarization	12
2.6 Conclusion	13
3. Analysis:.....	13
3.1. Introduction	13
3.2. Requirement Analysis (Survey)	14
3.3 Questionnaire Analysis	14
4. Implementation.	23
4.1. Introduction:	23
4.2. Dataset and Preprocessing	24
4.3. Model Creation.....	24
5. Results	35
5.1. Graphs and tables	36
5.2. Key Findings	40
6. Evaluation, Recommendations, and Conclusion.....	41
6.1. Evaluation	41
6.2. Recommendations.....	42
6.3. Conclusion.....	42
7. Project Management:	43
7.1 Milestones	43
7.2 Project Resources	43

7.3 Gantt Chart	44
7.4 Network Diagram	46
7.5 Work Breakdown Structure	48
8. References	49
Figure 1 Model_1 Summary (20k Dataset).....	25
Figure 2 Model_1 Summary (200k Dataset).....	26
Figure 3 Model_2 Summary (20k Dataset).....	26
Figure 4 Model_2 Summary (200k Dataset).....	27
Figure 5 Model_3 Summary (20k Dataset).....	28
Figure 6 Model_3 Summary (200k Dataset).....	28
Figure 7 Model_4 Summary (20k Dataset).....	29
Figure 8 Model_4 Summary (200k Dataset).....	30
Figure 9 Model_4 Structure.....	31
Figure 10 Model_5 Summary (20k Dataset).....	33
Figure 11 Model_5 Summary (200k Dataset).....	34
Figure 12 Model_5(Tribrid Model) Structure.....	35
Figure 13 Results 20k Dataset	36
Figure 14 Results 200k Dataset	36
Figure 15 Bar graphs of all models (evaluation metrics) 20k dataset.....	37
Figure 16 F-1 score comparison 20k Dataset.....	38
Figure 17 Bar graphs of all models (evaluation metrics) 200k dataset.....	39
Figure 18 F-1 score comparison 200k Dataset.....	40

Project Title: Developing AI Deep Learning Machine Learning Natural Language Processing Paper Skimming Model

Acknowledgement

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

As I complete this project, I feel immensely grateful to Allah who has given me the courage, determination and willingness to be able to bring this to a fruitful outcome. I want to extend my gratitude to my parents, my lecturers, who have equipped me with the skillset that has resulted in such a beautiful outcome and my friends for always believing and supporting me along the way.

I would also like to thank my supervisor, Dr. Manju Jose, for helping and guiding me through the span of this project, for supporting me and helping me learning and shape myself better under the light of her experiences and feedback. I want to also thank Dr. Senthil Ramadoss who has also been a great source of guidance for me in my time here at Majan University College.

Lastly, I want to thank and acknowledge the efforts that my brother has put in to help me get a better understanding of Artificial Intelligence. Thank you for being a guiding light for me.

May Allah shower his blessing on all of you.

Abstract

As scholarly publications currently experience exponential growth, researchers face a great challenge in the search for efficient ways of finding and synthesizing information from medical papers. To address this issue, this project proposes an AI driven Automated Paper Skimming System utilizing state-of-the-art deep learning and natural language processing (NLP) techniques. We combine token, character and positional embeddings into a Tribid Model to improve text classification. It extracts and summarizes key sections in an automated manner and lets researchers save time and get better results. The project was also shown to perform very well using the 20k and 200k RCT PubMed datasets. Our Tribid Model achieved F1-score of 82.86% and an accuracy of 82.97% with respect to the 20k dataset, which indicates its superiority in handling smaller datasets with a high accuracy. For the 200k dataset, the model was better at 85.2% accuracy and 85.12 F1 score. Finally, these results validate the scalability and robustness of the system with respect to the dataset size. In particular, the models were able to achieve state-of-the-art results by combining various multi-level embeddings, accounting for semantic as well as sequential dependencies, as well as acquiring structured insights from the unstructured abstracts. This work demonstrates the application of deep learning to biomedical research processes, presenting a structured, efficient, and high performing solution to the problem of information overload. These methodologies may be further refined in future implementations to fulfill a greater number of use cases beyond those currently applied in such a way as to better serve as integrated aspects within real world research settings.

1. Introduction

1.1. Project Background

Research papers are often seen as the primary source of information in numerous fields, presenting detailed information regarding scholars' findings, innovations, and ideas. New research papers are produced every day as many authors contribute to publishing their papers in different journals, conferences, and databases every year. Along with the growth of scientific knowledge, the number of articles, books, and publications increases steadily, which bare many benefits but also creates numerous difficulties for researchers and academics. The huge number of articles, papers, journals, and other publications means that a scholar needs to invest a lot of time and energy to keep up with the existing trends in one's discipline. Scholars stated that there is a pressure of having to read a number of papers and this pressure might be discouraging since it requires too much time to read them and even more time to synthesize the information obtained from them (*Tenopir et al., 2015; Bourne et al., 2015*).

In this regard, there is an especially high need for solutions that would allow a researcher to skim through medical papers to quickly identify relevant information. There are limitations in the conventional approaches of skimming through papers where we read through the papers looking for key features of the text which is more cumbersome and has the disadvantage of being commonly influenced by human subjectivity and inaccuracy (*Friedberg et al., 2018*). To address these challenges, this project proposes to design an automated System that uses deep learning models to skim on the medical papers in an efficient manner to make the process of research more efficient.

1.2. Project Introduction

The proposed project revolves around creating an Automated Skimming System that would use the advanced concepts of Deep Learning to skim through medical papers. Natural language processing (NLP) and machine learning (ML) will be used to review the research papers, identify sections of interest, and create summaries. The goal of the skimming system is thus to save the researchers' efforts and time and provide them with a concise understanding of several papers at once so that they could choose the most relevant ones for their projects (*Collobert et al., 2011*).

Deep learning, a subfield of machine learning, has become one of the most effective techniques to solve different problems in the sphere of NLP including text classification, sentiment analysis, and text summarizing. The fact that deep learning models are capable of capturing intricate patterns and contextual information from text makes them a good fit for the task of automated skimming through papers. This project will also attempt to determine the best deep learning architectures such as the Recurrent Neural Network (RNN), Convolutional Neural Network (CNN) and the Transformer Model on which the accurate and efficient skimming system would be based on. (*Devlin et al., 2018; Vaswani et al., 2017*).

1.3. Problem Statement

Original Skimming of medical papers is usually approached in a non-optimal way with significant disadvantages. It has been observed that researchers, contributors in the field of research, invest significant time in going through the papers, followed by summarizing them, which inherently reduces the overall time they get to spend in developing hypothesis and testing them. Furthermore, the manual method of skimming is more of a subjective one as different researcher may identify different key points from the same paper. Where the process is carried out subjectively, it can create variations in the information extracted hence determining the quality and credibility of synthesized research (*Nallapati et al., 2016*).

Moreover, the increasing number of published papers only amplifies the issue, whereby it becomes challenging for scholars to go through the latest publications. It also presents a strong demand for an automated system to filter the related literature by helping the researchers to quickly find parts of related papers that are beneficial without the need to spend much time (*See et al., 2017*).

1.4. Aim of Project

The main goal of this project is to implement a deep learning model for skimming the given medical papers automatically. The system should be able to perform the following tasks:

- Read through the content of medical papers with an aim of evaluating or categorizing its content to segment out the significant sections and further data.
- Present a clear synthesis of the information extracted as per the requirements of the problem in question.
- Ensure that the system is efficient and has an accuracy rate of above 80%.

To these ends, the project will help boost the effectiveness of literature review work, freeing up more time for researchers to focus on their key tasks.

1.5. Research Question

The central research question guiding this project is: 'How the deep learning algorithms can help to create an accurate and efficient system for skimming and summarizing the medical paper abstracts?' This question consists of a few sub-questions, namely:

- Which deep learning architectures show the best performance for text extraction and summarization?
- How can the applications of these models be trained and fine-tuned to achieve high levels of accuracy coupled with reduced computational times?
- What are the key problems and drawbacks of using automated paper skimming approach, how can they be minimized?

1.6. Project Objectives

The specific objectives of this project are as follows:

- 1) This paper aims to create a detailed system design plan that will show the major components and operations of the automated paper skimming system.
- 2) Develop and optimize deep learning algorithms for text extraction and text summarization to use the respective datasets of medical paper abstracts.
- 3) Preprocess and understand the dataset so that it can be processed, and our models can train on it.
- 4) Evaluate the developed system in terms of accuracy, time taken, and satisfaction of the user through testing and user consolation.
- 5) After the evaluation, the system should be able to make predictions on unseen data.

1.7. Scope of Research

The scope of this research encompasses several key areas, including:

- 1) **Literature Review:** Based on prior literature, analyze the current state of the technology and different algorithms used in the automated text summarization, deep learning in NLP, and the analysis of the medical papers. This review will assist in recognizing the areas that are not covered in previous studies and aid in designing the recommended system (*Chopra et al., 2016*).
- 2) **Dataset selection and Preprocessing:** Creating choosing an appropriate dataset and preprocessing it so that any medical paper abstract can be preprocess it according to the input needs of the models.
- 3) **Model Development:** Develop and deploy deep learning algorithms for extraction and as well as creation of summaries from texts. This will encompass choosing the right architectures of a model, data preprocessing to prepare the text inputs, and model training with huge collections of academic papers.
- 4) **Evaluation:** Carry out thorough tests that will help in determining how well the system works. This will include both the absolute/actual and relative/comparative numbers or proportions (quantitative measure) e.g. An evaluation strategy will involve the measures of validity (including accuracy, precision, and recall).
- 5) **Optimization:** Optimize the system based on the previous evaluations to improve the system success rate through model tuning, architectural enhancements, and solving all possible drawbacks.

2. Literature Review

2.1 Automated Text Summarization

Text summarization is a subfield of Natural Language Processing and is considered to be an important area in the specific field. It entails condensing documents into smaller texts while maintaining the core content, in a nutshell. There are two primary methodologies employed in text summarization, the concept extractive and abstractive summarization.

2.1.1 Extractive Summarization:

The extractive summarization techniques function by filtering essential sentence/sentences or mainly the phrases from the source text. These methods rank the percentage of occurrences of the words in those sentences by some parameters like term frequency- inverse document frequency (TF-IDF), position of the sentences, term frequency, specific key words in the sentences and so on. Graph-based models for the extractive methods, including TextRank that uses the PageRank algorithm and LexRank with cosine similarity applied to compute the centrality of each sentence (Mihalcea & Tarau, 2004; Erkan & Radev, 2004). These methods don't pose a lot of difficulty in the implementation of the summary and can produce summaries that are in correct English since they are made of actual English sentences taken from a number of textual sources. Nevertheless, extractive techniques deprive the summaries of coherence and context as the latter do not reword or paraphrase the textual content.

2.1.2 Abstractive Summarization:

Abstractive summarization refers to the process of coming up with new sentences that describe the content of a source text. This process is thus more involving than extractive summarization as it requires the user to both comprehend as well as paraphrase the texts. They use more complex forms of NLP such as Seq2Seq which is a model of the neural network that has both an encoder and decoder and use of attentiveness that makes the model to pay attention to constructive portions of the text in creation of summaries (Bahdanau et al., 2014). Sums such as the Transformer, which incorporated self-attention and enhanced the efficiency in NLP tasks, were helpful in abstractive summarization systems (Vaswani et al., 2017). Nevertheless, despite all the benefits that can be derived from them, the abstractive techniques can sometimes have problems with producing semantically correct summary sentence.

2.2 Deep Learning in Natural Language Processing

Deep learning has contributed significantly to advancing NLP whereby models that try to mimic human intelligence are created. Some of the deep learning architectures have played crucial role and have been instrumental in enhancing the text summarization as well as other NLP tasks.

2.2.1 Recurrent Neural Networks (RNNs):

RNNs are developed to handle such sequences thereby making it useful in text analysis. It keeps a hidden state which contains information from earlier steps and enables the model to discover relational dependencies between a sequence. RNNs have limitations such traditional RNNs, where the gradient can vanish over time, impairing the network's capacity to learn long-term dependence. Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU) was developed to overcome these problems by implementing structures that can regulate the flow of

data within the neural network and retain the gradient for long sequences (*Hochreiter and Schmidhuber, 1997; Cho et al., 2014*).

2.2.2 Convolutional Neural Networks (CNNs):

The CNNs are mainly used in image analysis but they are also applied in the NLP tasks. The CNNs are capable of seizing local pattern in textual data by applying convolutional filters across the input sequence. This helps the model to capture n-grams as well as other local patterns which makes it serve well in activities such as the classification of sentences as well as in extractive summarization activities. It has also been reported that CNNs perform well if max-pooling layers are added in the architecture to aid in decreasing the feature dimensionality and emphasizing the strongest features (*Kim, 2014*).

2.2.3 Transformer Models:

The use of the Transformer model in NLP was revolutionary and served as a great leap forward. While training, RNNs are sequential in nature, and they pass information from one time-step to other in a sequential manner, on the other hand, Transformers do not depend on sequential processing; they use self-attention so that all the time-steps can be processed in parallel. This enables the model to capture the global dependencies to a larger extent. Two transformer models are BERT (Bidirectional Encoder Representations from Transformers) and GPT (Generative Pre-trained Transformer, which were released recently and performed the state-of-art results in many NLP tasks including text summarization. BERT is especially suitable to identify contextual features within a sentence while GPT is best suitable for generative tasks involving language use such as language modeling and text generation to mention but a few (*Devlin et al., 2018; Radford et al., 2019*).

2.3 Tools and Technologies for Text Extraction and Summarization

There are many tools and technologies that exist to aid and facilitate the creation and implementation of text extraction and summarization systems. These tools include machine learning frameworks and specialized NLP libraries.

2.3.1 TensorFlow and Keras:

TensorFlow is machine learning platform developed by Google that houses and enables building of the deep learning models. These frameworks include TensorFlow combined with Keras that offers a higher-level programming interface for designing and testing neural networks. These frameworks provide a rich set of functionalities for developing models such as CNNs, RNNs, and even the more sophisticated transformers and have gained popularity among researchers and practitioners in artificial intelligence and Machine Learning (*Abadi et al., 2016*).

2.3.2 Natural Language Toolkit (NLTK) and SpaCy:

NLTK is a versatile structure intended for NLP and can be utilized to preprocess diverse composed records, tokenize content, parse strings, and perform semantic investigation. Especially, it applies to the learning context, and various brainstorming and designing processes

that require quick mock-up of an object or concept. Compared to that, SpaCy was developed for production and comes equipped with fast, baked-in models for most NLP tasks. Due to its emphasis on speed and user friendliness it is extensively used in creating complex NLP applications (*Bird et al., 2009; Honnibal & Montani, 2017*).

2.3.3 Pre-trained Models and Transfer Learning:

The use of pre-trained models has now greatly fostered the advancement of NLP. BERT, GPT-3, as are other models including T5 (Text-to-Text Transfer Transformer) and can be fine-tuned for any tasks with reasonably small samples. Fine-tuning and leveraging a model trained on one task for another is well established in the NLP field because of the high accuracy it achieves and the relatively less time needed for training, compared to training a new model from scratch (*Raffel et al., 2020*).

2.4 Existing Methodologies in Paper Analysis

Methods for analyzing and summarizing substantial portions of texts essentially depend on the structure and content of the volume of material one is working with, specifically academic papers in this case.

2.4.1 Citation-based Summarization:

This method utilizes the occurrences and the surroundings of citation in order to find out relative importance of certain parts of paper. Hence it can be said that highly cited sections are taken to be the ones which have a lot of substance to offer. Citation based summarization can also be of great importance in the understanding and determination of the key contributions and important works in the field (*Nanba & Okumura, 1999*).

2.4.2 Structure-based Summarization:

Scholarly articles present research findings in a certain style and are usually written in sections such as the abstract, introduction, method, result, and conclusion. Structure based summarization methods utilize this format for extracting information. For instance, the abstract and conclusion sections contain a lot of summarized information, and the conclusion section is normally densely packed with summaries, likewise, the methodology section avails a lot of information concerning research processes (*Teufel & Moens, 2002*).

2.4.3 Machine Learning-based Approaches:

In supervised learning, several models have been trained on labeled datasets, where the data items are annotated according to their relevancy; this is especially for particular sentences or parts of academic papers. Some of the algorithms used in these approaches include Support Vector Machines (SVM), Random Forests, and deep neural networks. These models learn how to search for and select crucial sentences for specific articles, which seminal work serve as the foundation for both extractive and abstractive summaries (*Cohan et al., 2018*).

2.5 Challenges and Limitations in Automated Summarization

Despite significant advancements, automated summarization faces several challenges that impact its effectiveness.

2.5.1 Semantic Understanding:

The problem of accurately reconstructing the semantics and context of the original text remains a challenge to this day. The limited grammar knowledge and inability to generate new sentences make grammatical error, continuity, and coherence issues common for abstractive summarization. They may yield syntactically sound but semantically deficient summaries that misrepresent the original text (*Kryściński et al., 2019*).

2.5.2 Data Quality and Availability:

The focus is put on the high-quality annotated datasets that are supposed to be used during the training of effective models. However, such datasets are often limited, particularly for niche use cases in a specific domain. One of the limitations is that the training set used to build the model is not rich in diversity and variety and can affect the outcomes of the model when applied to unseen documents (*Gupta & Lehal, 2010*).

2.5.3 Evaluation Metrics:

Automated summaries are relatively easy to come by, but it is challenging to establish criteria for assessing the quality of the summaries. Comparison is performed also in terms of percentage similarity between the generated summary and reference summaries by matching n-grams. Nevertheless, these metrics consider only the similarity in terms of string sequence and, therefore, may not provide an accurate view on the real semantic similarity and cohesion of the summaries. When it comes to human evaluation it may be accurate but firstly it is very time-consuming and secondly it may involve a lot of subjective factors (*Lin, 2004*).

2.5.4. Neural Networks for Joint Sentence Classification in Medical Paper Abstracts

This paper presents "Neural Networks for Joint Sentence Classification in Medical Paper Abstracts," a novel Artificial Neural Network (ANN) architecture for joint sentence classification in medical abstracts. Compared to conventional models with isolated sentence classification, the envisioned approach considers contextual information by concurrently classifying sentences in a sequence. Current methods including Naive Bayes, Support Vector Machines (SVMs), and Conditional Random Fields (CRFs) do not deal very well with sequence dependencies and require hand crafted features. In contrast, the proposed model utilizes a hybrid token enforcement layer to combine token and character embeddings to produce enriched semantic representation. Further, it incorporates a label sequence optimization layer to exploit dependencies between sentence labels and exploits a bidirectional Long Short-Term Memory (bi-LSTM) network to encode sentence features. Coherent classification of abstract section objectives, methods, results and conclusions is ensured by this structured prediction method.

The model was evaluated on two medical abstract datasets: Both PubMed 20k RCT and NICTA-PIBOSO, annotated for sequential sentence classification. Pag. 3 spec. mentions 20k abstracts

from the PubMed 20k RCT dataset with 68k vocabulary size and five sentence labels (objectives, background, methods, results, and conclusions), trained on 15k abstracts (195k sentences), validation on 2.5k abstracts (33k sentences) and test on 2.5k abstracts (33k sentences). Although smaller it contains 999 abstracts with vocabulary size 17k and six sentence labels. It is broken into 722 abstracts (8k sentences) for training, 77 abstracts (0.9k sentences) for validation and 200 abstracts (2k sentences) for testing. Despite varying in dataset size and complexity, the proposed model matches state-of-the-art performance for both benchmarks, outperforming CRFs and some previous ANN based approaches.

The results demonstrate that the proposed ANN model excels due to several key factors: its elimination of human engineered features, use of embeddings rather than n-grams, and joint learning of a sentence label prediction and an embedding. For instance, the model gets an F1 score of 89.9% in the case of the PubMed dataset highlighting the model's robustness in precision as well as recall. Coherence in sentence classification is further enhanced by the model's ability to predict labels jointly, making it especially suitable for structured information retrieval from unstructured abstracts. This advancement is important in the biomedical domain in particular, where the exponential volume of scholarly articles makes it ever harder for researchers to quickly find efficient access to relevant information. With the ambitions of automating, optimizing and even accelerating sentence classification and increasing accuracy, this model can bring transformations in tasks such as information retrieval, summarization, and evidence extraction in biomedical research. (Dernoncourt, Lee and Szolovits, 2016)

2.6 Conclusion

The literature review shows a significant level of advancement in the automating of text and text summarization and the use of deep learning in NLP. From the two kinds of summarization the extractive kind has a generally simple way of pinpointing the relevant sentences to be selected while the abstract has the flexibility of creating new information. Current transformer models have also improved the effectiveness of NLP systems in terms of more adequate and context-related summarization.

However, there are certain issues that are crucial in converting also warranted future research, including semantic understanding, quality of data, and there is a need for growth of good metrics for evaluation. Meeting these challenges will be crucial for the further development of technology in terms of automated summarization and building useful tools for assisting researchers facing the difficulties of intense influx of newly published articles.

3. Analysis:

3.1. Introduction

This chapter describes the procedure used in acquiring relevant data on the development of the Automated Paper Skimming Tool. After this, it will proceed to the examination of the data that was gathered through the application of the outlined methods and techniques.

3.2. Requirement Analysis (Survey)

The analyzed outcomes are then summarized to provide a comprehensive view of the data collection phase. Data gathering techniques fall into two categories, namely, the qualitative research and the quantitative research. This study uses a quantitative approach and method to collect data through a structured questionnaire. This enabled the establishment of this questionnaire for the purpose of testing qualitative hypothesis on the user requirements and preference.

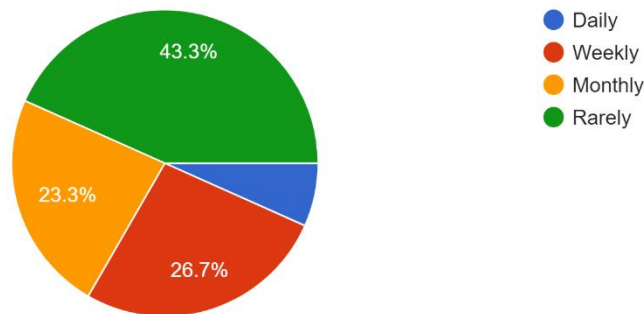
Consequently, the survey was conducted using Google Forms. The sample consisted of thirty to thirty-five individuals, and they were 18 years and above, 55+ years. The questionnaire focused on the users who are actively engaged in academic research, including students, researchers, and professors, to make the usage more diversified. That is using the online format of the questionnaire, it was easier to appreciate that the respondents were freely and willingly filling the questionnaires at any one time they felt free to do so. More to the point, the respondents' anonymity made it possible to solicit candid feedback. These questions were developed in accordance with the research aims, which required collecting information about the difficulties, requirements, and preferences expressed by respondents regarding the skimming of academic articles. This collected information would be a significant contribution to the enhancement and furtheration of the Automated Paper Skimming Tool.

3.3 Questionnaire Analysis

Question 1

How often do you read academic papers?

30 responses



Daily	2	6.667	6.667	6.667
Monthly	7	23.333	23.333	30.000
Rarely	13	43.333	43.333	73.333

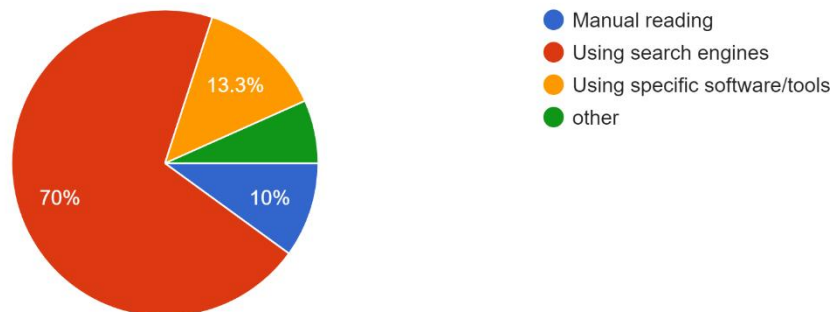
Weekly	8	26.667	26.667	100.000
Missing	0	0.000		
Total	30	100.000	100.000	100.000

Analysis: It is indicated that majority of the respondents read the academic papers occasionally and rarely the figures being 43.33% while 26.67% reported that they read the papers on weekly bases. This implies that both the frequent and the infrequent reader must be considered so that their productivity can be enhanced.

Question 2

How do you currently find relevant information in academic papers?

30 responses

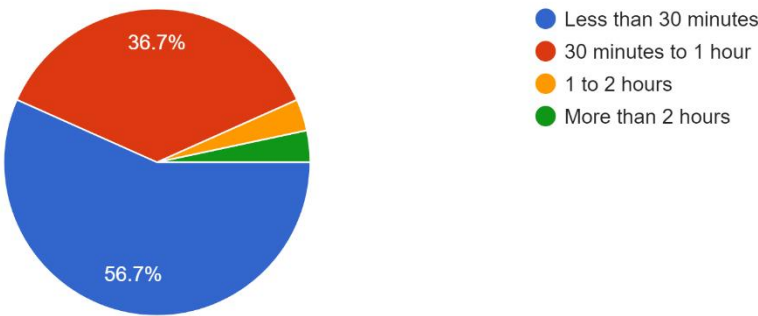


Manual reading	3	10.000	10.000	10.000
Using search engines	21	70.000	70.000	80.000
Using specific software/tools	4	13.333	13.333	93.333
Other	2	6.667	6.667	100.000
Missing	0	0.000		
Total	30	100.000	100.000	100.000

Analysis: Overall, the respondents' responses show that a healthy 70% of the sources relied on search-engines, indicating a bias towards the fully automated or semi-automated tools. This becomes a plus for the tool of skimming papers and therefore underlines the importance of incorporating search and retrieval functionality in the program.

Question 3

How long does it typically take you to find relevant information in a paper?
30 responses

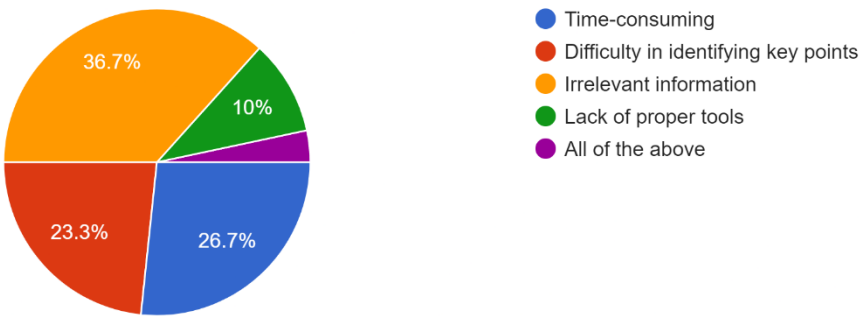


1 to 2 hours	1	3.333	3.333	3.333
30 minutes to 1 hour	11	36.667	36.667	40.000
Less than 30 minutes	17	56.667	56.667	96.667
More than 2 hours	1	3.333	3.333	100.000
Missing	0	0.000		
Total	30	100.000	100.000	100.000

Analysis: The youngest (56, 67%) can cover the topics within half an hour while the others; 40% of people take half an hour to 2 hours on average. The specified tool must be able to minimize the amount of time spent actually skimming through the papers as much as possible.

Question 4

What are the main challenges you face when skimming through academic papers?
30 responses



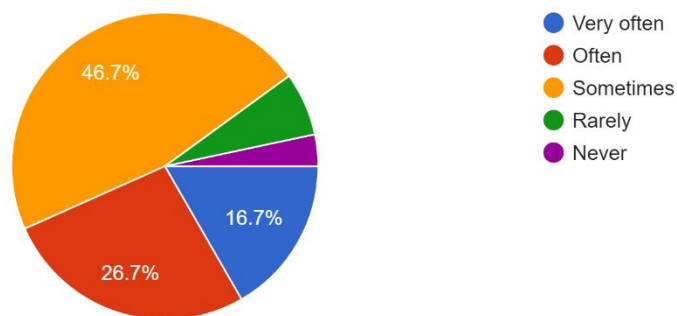
All of the above	1	3.333	3.333	3.333
Difficulty in identifying key points	7	23.333	23.333	26.667
Irrelevant information	11	36.667	36.667	63.333
Lack of proper tools	3	10.000	10.000	73.333
Time-consuming	8	26.667	26.667	100.000
Missing	0	0.000		
Total	30	100.000	100.000	100.000

Analysis: Largest obstacle is accidentally coming across something not suitable for the search or task (36.67%) and that it takes precious time to look for reliable information (26.67%). I think that the tool should be more focused on pointing out and dividing material into significant aspects and lower its concern with the actual text content.

Question 5

How often do you encounter irrelevant information in the papers you read?

30 responses



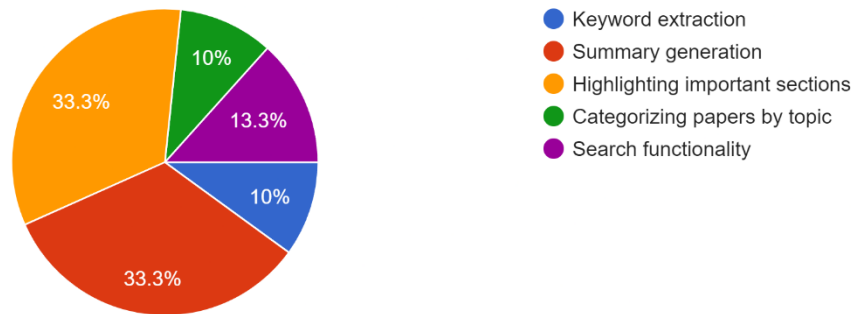
Never	1	3.333	3.333	3.333
Often	8	26.667	26.667	30.000
Rarely	2	6.667	6.667	36.667
Sometimes	14	46.667	46.667	83.333
Very often	5	16.667	16.667	100.000
Missing	0	0.000		
Total	30	100.000	100.000	100.000

Analysis: While 46.67% said they come across irrelevant information occasionally, a significantly smaller percentage of them 26.67% encounter it often. The tool needs present rich relevance filtering to enhance the readability of the given text by eliminating the most unnecessary contents.

Question 6

Which feature do you find most useful in an automated paper skimming tool?

30 responses



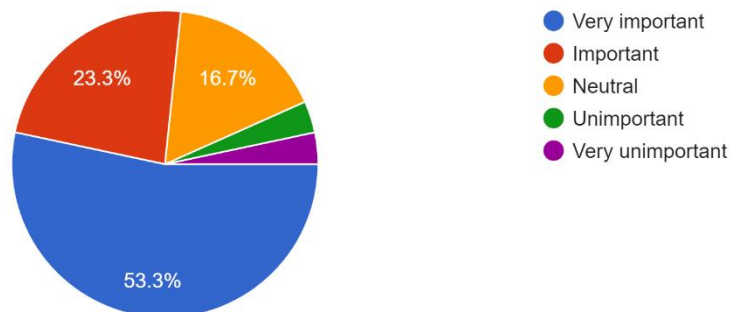
Categorizing papers by topic	3	10.000	10.000	10.000
Highlighting important sections	10	33.333	33.333	43.333
Keyword extraction	3	10.000	10.000	53.333
Search functionality	4	13.333	13.333	66.667
Summary generation	10	33.333	33.333	100.000
Missing	0	0.000		
Total	30	100.000	100.000	100.000

Analysis: The following features are the most required tools and each has been noted to be required by 33 educators and trainers: 33% of respondents. The tool has to be designed to address all these functionalities in a way that will ensure it is a solution to the problem as perceived by the users.

Question 7

How important is the accuracy of information extraction for you?

30 responses



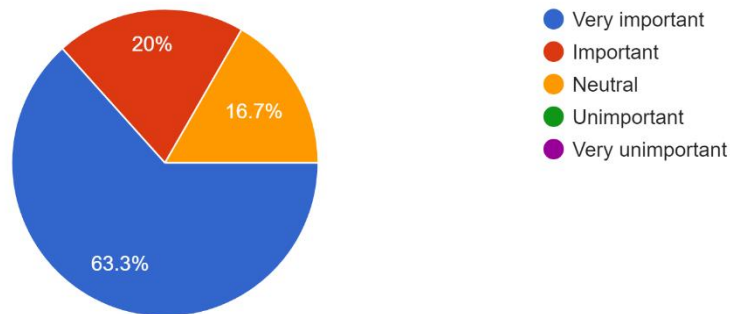
Important	7	23.333	23.333	23.333
Neutral	5	16.667	16.667	40.000
Unimportant	1	3.333	3.333	43.333
Very important	16	53.333	53.333	96.667
Very unimportant	1	3.333	3.333	100.000
Missing	0	0.000		
Total	30	100.000	100.000	100.000

Analysis: Accuracy is crucial, with 76.66% rating it as very important or important. The tool must ensure high accuracy in extracting relevant information.

Question 8

How important is the speed of information extraction for you?

30 responses



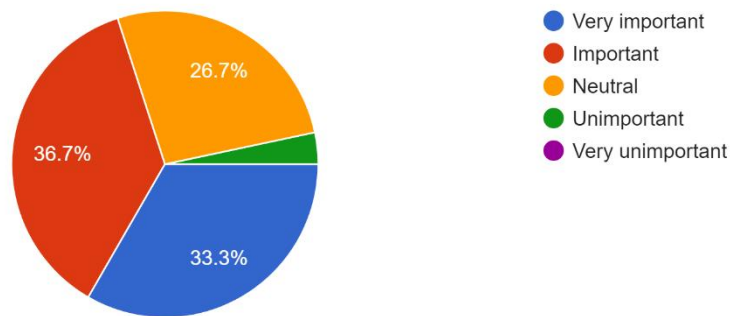
Important	6	20.000	20.000	20.000
Neutral	5	16.667	16.667	36.667
Very important	19	63.333	63.333	100.000
Missing	0			
Total	30	100.000	100.000	100.000

Analysis: Speed is a key factor, with 83.33% rating it as very important or important. The tool should be optimized for fast processing.

Question 9

How beneficial would customization options for different academic disciplines be for you?

30 responses



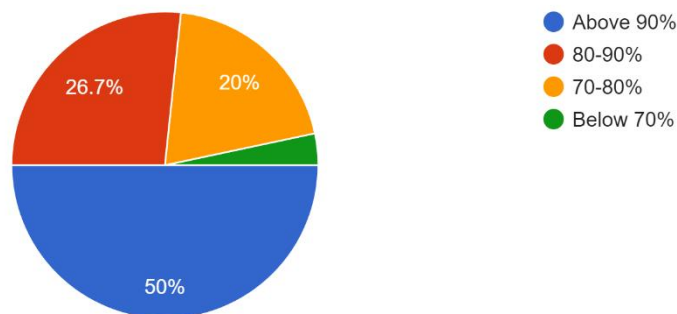
Important	11	36.667	36.667	36.667
Neutral	8	26.667	26.667	63.333
Unimportant	1	3.333	3.333	66.667
Very important	10	33.333	33.333	100.000
Missing	0	0.000		
Total	30	100.000	100.000	100.000

Analysis: Options are highly valued with regards to customization, 70% of the participants rating the options as important or very important. Having the option for work-specific customizations will improve the application of the tool across the various disciplines.

Question 10

What level of accuracy do you expect from an automated paper skimming tool?

30 responses



70-80%	6	20.000	20.000	20.000

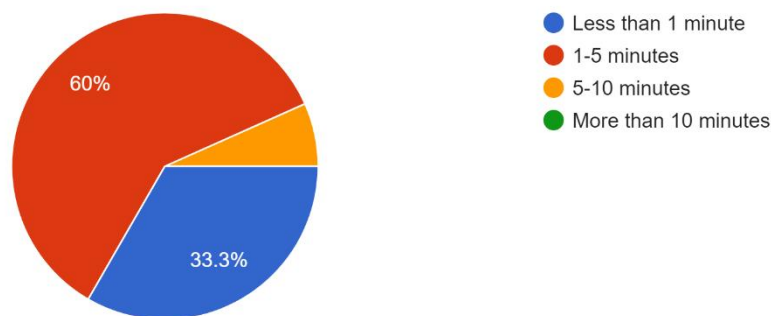
80-90%	8	26.667	26.667	46.667
Above 90%	15	50.000	50.000	96.667
Below 70%	1	3.333	3.333	100.000
Missing	0	0.000		
Total	30	100.000	100.000	100.000

Analysis: The expected level of accuracy is not very low with a reported average of 50% expecting over 90% accuracy. These expectations should be met by the tool and it should be designed to have high levels of accuracy in order to offer the right details.

Question 11

How quickly do you expect the tool to skim and provide information from a paper?

30 responses



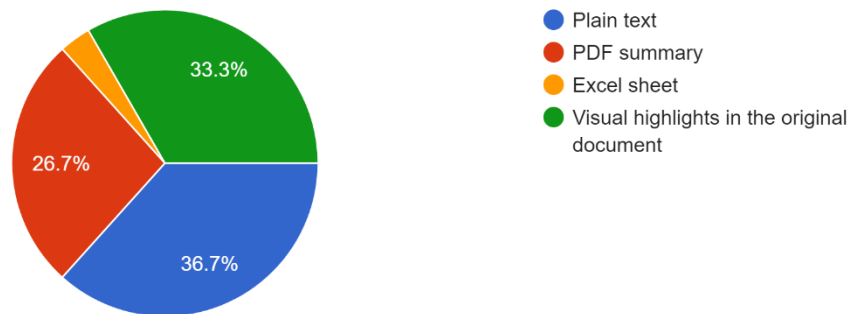
1-5 minutes	18	60.000	60.000	60.000
5-10 minutes	2	6.667	6.667	66.667
Less than 1 minute	10	33.333	33.333	100.000
Missing	0	0.000		
Total	30	100.000	100.000	100.000

Analysis: About 60% of respondents believe that outcomes do not take more than 5 to 1 minutes. 33.33% of the respondents said they expect it in 5-10 minutes, 22% said in 1-5 minutes, and 16% said less than 1 minute. To meet the expectations of the users, the tool should strive to make quick work of the analytics.

Question 12

Which type of output format do you prefer for the extracted information?

30 responses



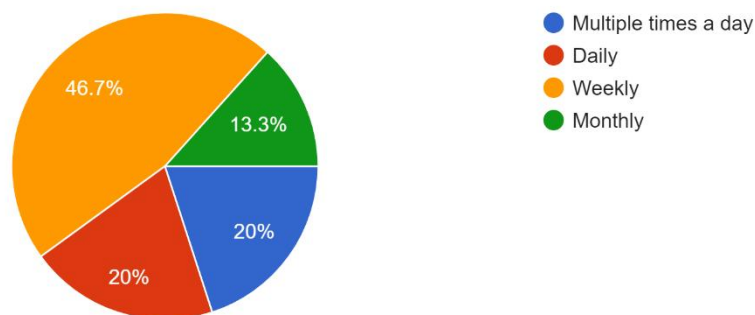
Excel sheet	1	3.333	3.333	3.333
PDF summary	8	26.667	26.667	30.000
Plain text	11	36.667	36.667	66.667
Visual highlights in the original document	10	33.333	33.333	100.000
Missing	0	0.000		
Total	30	100.000	100.000	100.000

Analysis: Concerning the output format, simple text is preferred with highlighted visual components acting as the second choice. This should allow the tool to generate its output in different formats depending on the user's preference.

Question 13

How frequently do you expect to use an automated paper skimming tool?

30 responses



Daily	6	20.000	20.000	20.000

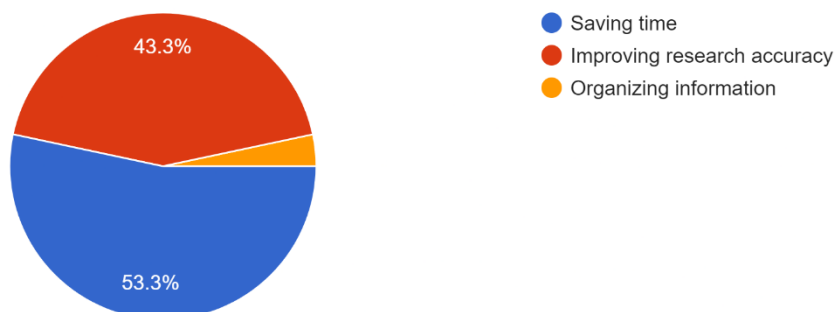
Monthly	4	13.333	13.333	33.333
Multiple times a day	6	20.000	20.000	53.333
Weekly	14	46.667	46.667	100.000
Missing	0	0.000		
Total	30	100.000	100.000	100.000

Analysis: The tool's forecasted usage frequently indicates a high percentage of 66.67% of respondents stated that they employed the technology daily or weekly. It should be a working press primarily intended for practical everyday use and the overriding priority should be durability and speed.

Question 14

What is your primary goal in using an automated paper skimming tool?

30 responses



Improving research accuracy	13	43.333	43.333	43.333
Organizing information	1	3.333	3.333	46.667
Saving time	16	53.333	53.333	100.000
Missing	0	0.000		
Total	30	100.000	100.000	100.000

Analysis: The primary objectives first include time saving which is 53.33% and, secondly, more reliable research information which is 43.33%. It should be aimed at these aspects so that it will be more aligned to the user expectations and useful in the way that it will bring improvements in terms of efficiency and accuracy.

4. Implementation.

4.1. Introduction:

With more than 50 million articles published to date in scholarly biomedical literature, the exponential growth in volume has been problematic in terms of efficiently extracting critical information. Structured abstracts (i.e. organized into sections like Objective, Method, Results,

and Conclusion) provide some simplification of this process, yet many abstracts are not structured. This project aims to solve this issue by sequential sentence classification, i.e. automatically labelling of sentences within medical abstracts according to their respective roles. And this approach can categorize sentences into predefined labels (Objective, Method, Results), helping researchers carry out tasks such as text summarization and information extraction to speed up and increase the accuracy of skimming medical literature.

While unique in implementation, this project was motivated by the work with “Neural Networks for Joint Sentence Classification in Medical Paper Abstracts” by Dernoncourt, Lee, and Szolovits (2016). Their model revealed the utility of token embeddings combined with character embeddings and structured prediction on a sequential sentence classification problem in medical abstracts. Based on their work, this project reimagines and improves their model, which can classify sentences into pre-defined roles such as Objective, Method, Results, and Conclusion.

4.2. Dataset and Preprocessing

This project uses the PubMed 200k RCT Dataset which is a labeled corpus of abstracts from randomized controlled trials that this project utilizes.. Each abstract sentence is assigned to one of five categories: **Background, Objective, Methods, Results, and Conclusions**. Sequential sentence classification is dependent on the dataset as it works as a medium to support organizing unstructured abstracts which can be skimmed more easily by researchers.

4.2.1. Preprocessing Steps:

1. **Tokenization:** Multi-level feature extraction was obtained by splitting sentences into tokens and characters.
2. **Embedding Preparation:** Pre-trained embeddings such as **GloVe** and **Universal Sentence Encoder (USE)** were utilized for richer text representations.
3. **Positional Information:** Sentence sequence was maintained by adding line numbers.
4. **Text Cleaning:** Symbols and punctuation were removed for consistency.

4.3. Model Creation

There are 2 datasets in this PubMed 200k RCT Dataset. One is 20k and the other one is 200k. I have made models on both so as we get into the implementation and look at the model summary we will be able to see some differences even though the same approach is followed for both models.

4.3.1. Model 0: Baseline - TF-IDF + Naive Bayes

The baseline model uses TF-IDF (Term Frequency-Inverse Document Frequency) for feature extraction, and Multinomial Naive Bayes for classification.

- **Purpose:** Setting a performance benchmark using a simple, straight forward approach.
- **Architecture:**
 - Input: The cleaned text converted to TF-IDF features.
 - Process: Sentence categories are predicted by Naive Bayes classifier.
- **Tools:** TF-IDF and MultinomialNB of Scikit-learn.

4.3.2. Model 1: Conv1D with Token Embeddings

The 1D Convolutional Neural Network for token level sentence classification is incorporated into this model.

- **Purpose:** To explore how token embeddings and convolutional layers can be used in tandem to obtain features..
- **Architecture:**
 - **Embedding Layer:** Tokens are mapped to 300-dimensional GloVe embeddings.
 - **Conv1D Layer:** It extracts local patterns from a sequence of token.
 - **Global Max Pooling:** It reduces feature maps to a single vector.
 - **Dense Layer:** It predicts sentence class probabilities.
- **Tools:** Conv1D implementation using TensorFlow / Keras.
- **Model_1 summary:**

```
# Get summary of Conv1D model  
model_1.summary()
```

Model: "functional_8"

Layer (type)	Output Shape	Param #
input_layer_3 (InputLayer)	(None, 1)	0
text_vectorization_1 (TextVectorization)	(None, 55)	0
token_embedding (Embedding)	(None, 55, 128)	8,299,648
conv1d_2 (Conv1D)	(None, 55, 64)	41,024
global_max_pooling1d_2 (GlobalMaxPooling1D)	(None, 64)	0
dense_10 (Dense)	(None, 5)	325

Total params: 8,340,997 (31.82 MB)
Trainable params: 8,340,997 (31.82 MB)
Non-trainable params: 0 (0.00 B)

Figure 1 Model_1 Summary (20k Dataset)

```
# Get summary of Conv1D model
model_1.summary()
```

Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 1)	0
text_vectorization_1 (TextVectorization)	(None, 55)	0
token_embedding (Embedding)	(None, 55, 128)	42,368,000
conv1d_1 (Conv1D)	(None, 55, 64)	41,024
global_max_pooling1d_1 (GlobalMaxPooling1D)	(None, 64)	0
dense_1 (Dense)	(None, 5)	325

Total params: 42,409,349 (161.78 MB)
 Trainable params: 42,409,349 (161.78 MB)
 Non-trainable params: 0 (0.00 B)

Figure 2 Model_1 Summary (200k Dataset)

4.3.3. Model 2: Feature Extraction with Pretrained Token Embeddings

Pretrained token embeddings for Feature Extraction Feature extraction was carried out using token embeddings that were pre trained without any further training.

- **Purpose:** In order to evaluate pre trained embeddings such as GloVe without fine tuning.
- **Architecture:**
 - Input: Sentences tokenized and represented with GloVe embeddings.
 - Dense layers classify the sentence embeddings.
- **Model_2 summary:**

```
# Display model summary
model_2.summary()
```

Model: "text_classifier"

Layer (type)	Output Shape	Param #
text_input (InputLayer)	(None)	0
universal_sentence_encoder_layer_5 (UniversalSentenceEncoderLayer)	(None, 512)	0
dense_11 (Dense)	(None, 128)	65,664
output (Dense)	(None, 5)	645

Total params: 66,309 (259.02 KB)
 Trainable params: 66,309 (259.02 KB)
 Non-trainable params: 0 (0.00 B)

Figure 3 Model_2 Summary (20k Dataset)

```
# Display model summary
model_2.summary()
```

Model: "text_classifier"

Layer (type)	Output Shape	Param #
text_input (InputLayer)	(None)	0
universal_sentence_encoder_layer (UniversalSentenceEncoderLayer)	(None, 512)	0
dense_2 (Dense)	(None, 128)	65,664
output (Dense)	(None, 5)	645

Total params: 66,309 (259.02 KB)
 Trainable params: 66,309 (259.02 KB)
 Non-trainable params: 0 (0.00 B)

Figure 4 Model_2 Summary (200k Dataset)

4.3.4 Model 3: Conv1D with Character Embeddings

This model extends the Conv1D architecture by incorporating **character embeddings**, which handle out-of-vocabulary words and spelling variations.

- **Purpose:** To capture fine-grained features at the character level.
- **Architecture:**
 - **Character Embedding Layer:** Characters mapped to 25-dimensional vectors.
 - **Conv1D Layer:** Detects patterns in character sequences.
 - **Pooling + Dense Layer:** Aggregates features and predicts class labels.
- **Model_3 summary:**

```
# Check the summary of conv1d_char_model
model_3.summary()
```

Model: "model_3_conv1D_char_embedding"

Layer (type)	Output Shape	Param #
input_layer_4 (InputLayer)	(None, 1)	0
char_vectorizer (TextVectorization)	(None, 290)	0
char_embed (Embedding)	(None, 290, 25)	1,750
conv1d_3 (Conv1D)	(None, 290, 64)	8,064
global_max_pooling1d_3 (GlobalMaxPooling1D)	(None, 64)	0
dense_12 (Dense)	(None, 5)	325

Total params: 10,139 (39.61 KB)
Trainable params: 10,139 (39.61 KB)
Non-trainable params: 0 (0.00 B)

Figure 5 Model_3 Summary (20k Dataset)

```
# Check the summary of conv1d_char_model
model_3.summary()
```

Model: "model_3_conv1D_char_embedding"

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 1)	0
char_vectorizer (TextVectorization)	(None, 291)	0
char_embed (Embedding)	(None, 291, 25)	1,750
conv1d_2 (Conv1D)	(None, 291, 64)	8,064
global_max_pooling1d_2 (GlobalMaxPooling1D)	(None, 64)	0
dense_3 (Dense)	(None, 5)	325

Total params: 10,139 (39.61 KB)
Trainable params: 10,139 (39.61 KB)
Non-trainable params: 0 (0.00 B)

Figure 6 Model_3 Summary (200k Dataset)

4.3.5. Model 4: Hybrid Model - Pretrained Token Embeddings + Character Embeddings

This model combines **token embeddings** and **character embeddings** to create a hybrid sentence representation.

- **Purpose:** To leverage the strengths of both token-level and character-level embeddings.
- **Architecture:**
 - **Token Embedding Layer:** GloVe embeddings for word-level semantics.

- **Character Embedding Layer:** Character-level features via Conv1D.
- **Concatenation Layer:** Combines outputs of token and character embeddings.
- **Dense Layer:** Predicts sentence classes.
- **Model_4 summary:**

```
# Get summary of token and character model
model_4.summary()
```

Model: "model_4_token_and_char_embeddings"

Layer (type)	Output Shape	Param #	Connected to
char_input (InputLayer)	(None, 1)	0	-
token_input (InputLayer)	(None)	0	-
char_vectorizer (TextVectorization)	(None, 290)	0	char_input[0][0]
universal_sentence_encod... (UniversalSentenceEncode...	(None, 512)	0	token_input[0][0]
char_embed (Embedding)	(None, 290, 25)	1,750	char_vectorizer[1][0]
token_output (Dense)	(None, 128)	65,664	universal_sentence_en...
bidirectional_2 (Bidirectional)	(None, 50)	10,200	char_embed[1][0]
token_char_hybrid (Concatenate)	(None, 178)	0	token_output[0][0], bidirectional_2[0][0]
dropout_3 (Dropout)	(None, 178)	0	token_char_hybrid[0][...]
dense_13 (Dense)	(None, 200)	35,800	dropout_3[0][0]
dropout_4 (Dropout)	(None, 200)	0	dense_13[0][0]
dense_14 (Dense)	(None, 5)	1,005	dropout_4[0][0]

Total params: 114,419 (446.95 KB)
Trainable params: 114,419 (446.95 KB)
Non-trainable params: 0 (0.00 B)

Figure 7 Model_4 Summary (20k Dataset)

```
# Get summary of token and character model
model_4.summary()
```

Model: "model_4_token_and_char_embeddings"

Layer (type)	Output Shape	Param #	Connected to
char_input (InputLayer)	(None, 1)	0	-
token_input (InputLayer)	(None)	0	-
char_vectorizer (TextVectorization)	(None, 291)	0	char_input[0][0]
universal_sentence_encod... (UniversalSentenceEncode...	(None, 512)	0	token_input[0][0]
char_embed (Embedding)	(None, 291, 25)	1,750	char_vectorizer[1][0]
token_output (Dense)	(None, 128)	65,664	universal_sentence_en...
bidirectional (Bidirectional)	(None, 50)	10,200	char_embed[1][0]
token_char_hybrid (Concatenate)	(None, 178)	0	token_output[0][0], bidirectional[0][0]
dropout (Dropout)	(None, 178)	0	token_char_hybrid[0][...
dense_4 (Dense)	(None, 200)	35,800	dropout[0][0]
dropout_1 (Dropout)	(None, 200)	0	dense_4[0][0]
dense_5 (Dense)	(None, 5)	1,005	dropout_1[0][0]

Total params: 114,419 (446.95 KB)
Trainable params: 114,419 (446.95 KB)
Non-trainable params: 0 (0.00 B)

Figure 8 Model_4 Summary (200k Dataset)

- **Model_4 Structure:**

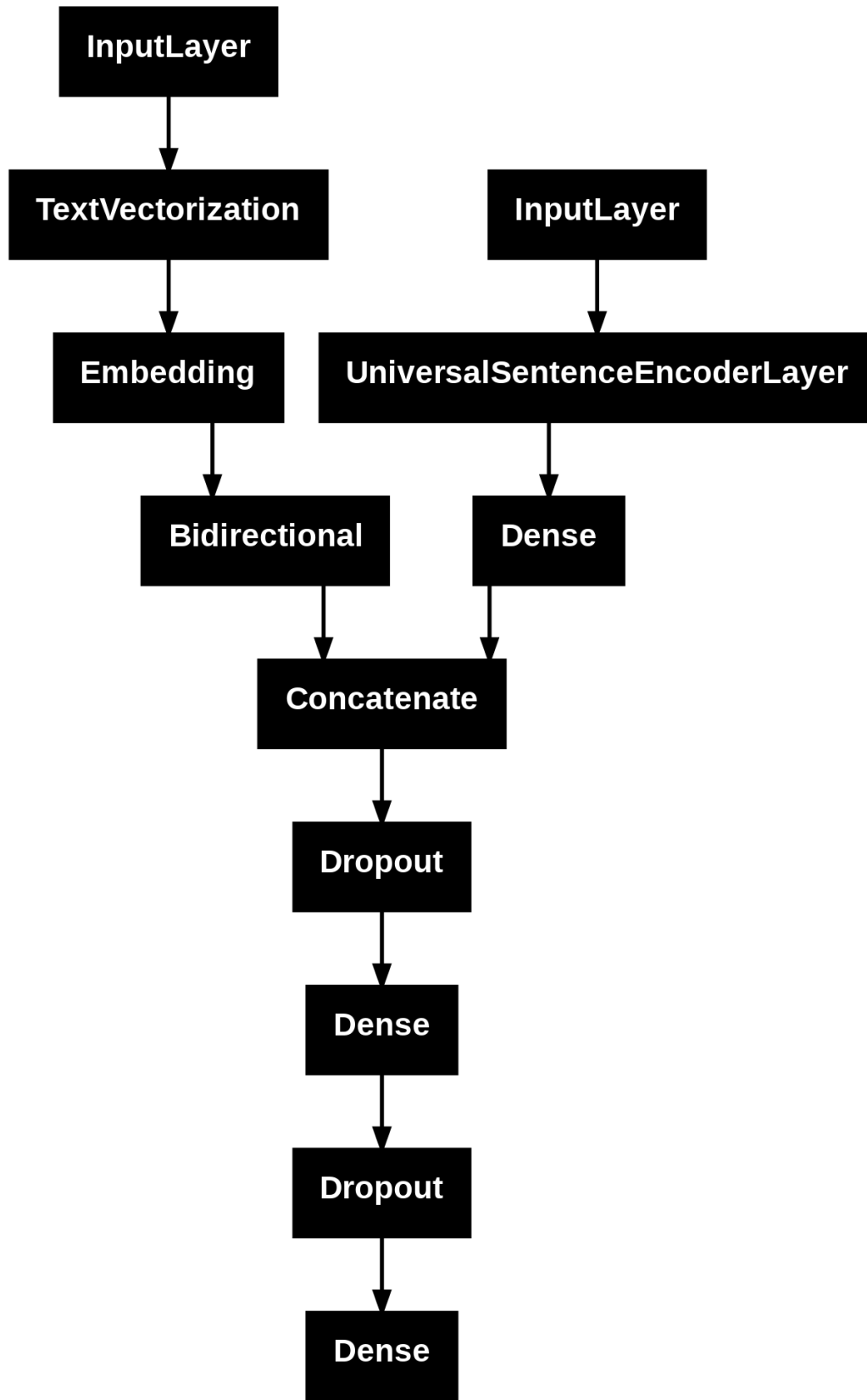


Figure 9 Model_4 Structure

4.3.6. Final Model, Model 5: Transfer Learning with Pretrained Token Embeddings, Character Embeddings, and Positional Embeddings

The final model integrates three embedding strategies: We apply transfer learning principles, using token, character, and positional embeddings.

- **Purpose:** For combining advanced features for the best performance.
- **Architecture:**
 - **Token Embeddings:** Pre-trained GloVe embeddings.
 - **Character Embeddings:** Captured through Conv1D.
 - **Positional Embeddings:** Encoding sentence positions to preserve information regarding the sequences.
 - **Fusion Layer:** All three embeddings are concatenated.
 - **Bidirectional LSTM:** All of them are concatenated to get all embeddings..
 - **Dense Layer:** Final prediction layer.
- **Model_5 summary:**

```
# Get a summary of our token, char and positional embedding model
model_5.summary()
```

Model: "functional_15"

Layer (type)	Output Shape	Param #	Connected to
char_inputs (InputLayer)	(None, 1)	0	-
token_input (InputLayer)	(None)	0	-
char_vectorizer (TextVectorization)	(None, 290)	0	char_inputs[0][0]
universal_sentence_encod... (UniversalSentenceEncode...	(None, 512)	0	token_input[0][0]
char_embed (Embedding)	(None, 290, 25)	1,750	char_vectorizer[2][0]
token_outputs (Dense)	(None, 128)	65,664	universal_sentence_en...
bidirectional_3 (Bidirectional)	(None, 64)	14,848	char_embed[2][0]
token_char_hybrid_embdd... (Concatenate)	(None, 192)	0	token_outputs[0][0], bidirectional_3[0][0]
line_number_input (InputLayer)	(None, 15)	0	-
total_lines_input (InputLayer)	(None, 20)	0	-
dense_17 (Dense)	(None, 256)	49,408	token_char_hybrid_emb...
dense_15 (Dense)	(None, 32)	512	line_number_input[0][...
dense_16 (Dense)	(None, 32)	672	total_lines_input[0][...
dropout_5 (Dropout)	(None, 256)	0	dense_17[0][0]
token_char_positional_em... (Concatenate)	(None, 320)	0	dense_15[0][0], dense_16[0][0], dropout_5[0][0]
output_layer (Dense)	(None, 5)	1,605	token_char_positional...

Total params: 134,459 (525.23 KB)

Trainable params: 134,459 (525.23 KB)

Non-trainable params: 0 (0.00 B)

Figure 10 Model_5 Summary (20k Dataset)

```
# Get a summary of our token, char and positional embedding model
model_5.summary()
```

Model: "functional_8"

Layer (type)	Output Shape	Param #	Connected to
char_inputs (InputLayer)	(None, 1)	0	-
token_input (InputLayer)	(None)	0	-
char_vectorizer (TextVectorization)	(None, 291)	0	char_inputs[0][0]
universal_sentence_encod... (UniversalSentenceEncode...	(None, 512)	0	token_input[0][0]
char_embed (Embedding)	(None, 291, 25)	1,750	char_vectorizer[2][0]
token_outputs (Dense)	(None, 128)	65,664	universal_sentence_en...
bidirectional_1 (Bidirectional)	(None, 64)	14,848	char_embed[2][0]
token_char_hybrid_embddd... (Concatenate)	(None, 192)	0	token_outputs[0][0], bidirectional_1[0][0]
line_number_input (InputLayer)	(None, 15)	0	-
total_lines_input (InputLayer)	(None, 20)	0	-
dense_8 (Dense)	(None, 256)	49,408	token_char_hybrid_emb...
dense_6 (Dense)	(None, 32)	512	line_number_input[0][...
dense_7 (Dense)	(None, 32)	672	total_lines_input[0][...
dropout_2 (Dropout)	(None, 256)	0	dense_8[0][0]
token_char_positional_em... (Concatenate)	(None, 320)	0	dense_6[0][0], dense_7[0][0], dropout_2[0][0]
output_layer (Dense)	(None, 5)	1,605	token_char_positional...

Total params: 134,459 (525.23 KB)
Trainable params: 134,459 (525.23 KB)
Non-trainable params: 0 (0.00 B)

Figure 11 Model_5 Summary (200k Dataset)

- **Model_5 Structure:**

```
[ ] # Plot the token, char, positional embedding model
from tensorflow.keras.utils import plot_model
plot_model(model_5, show_shapes=True)
```

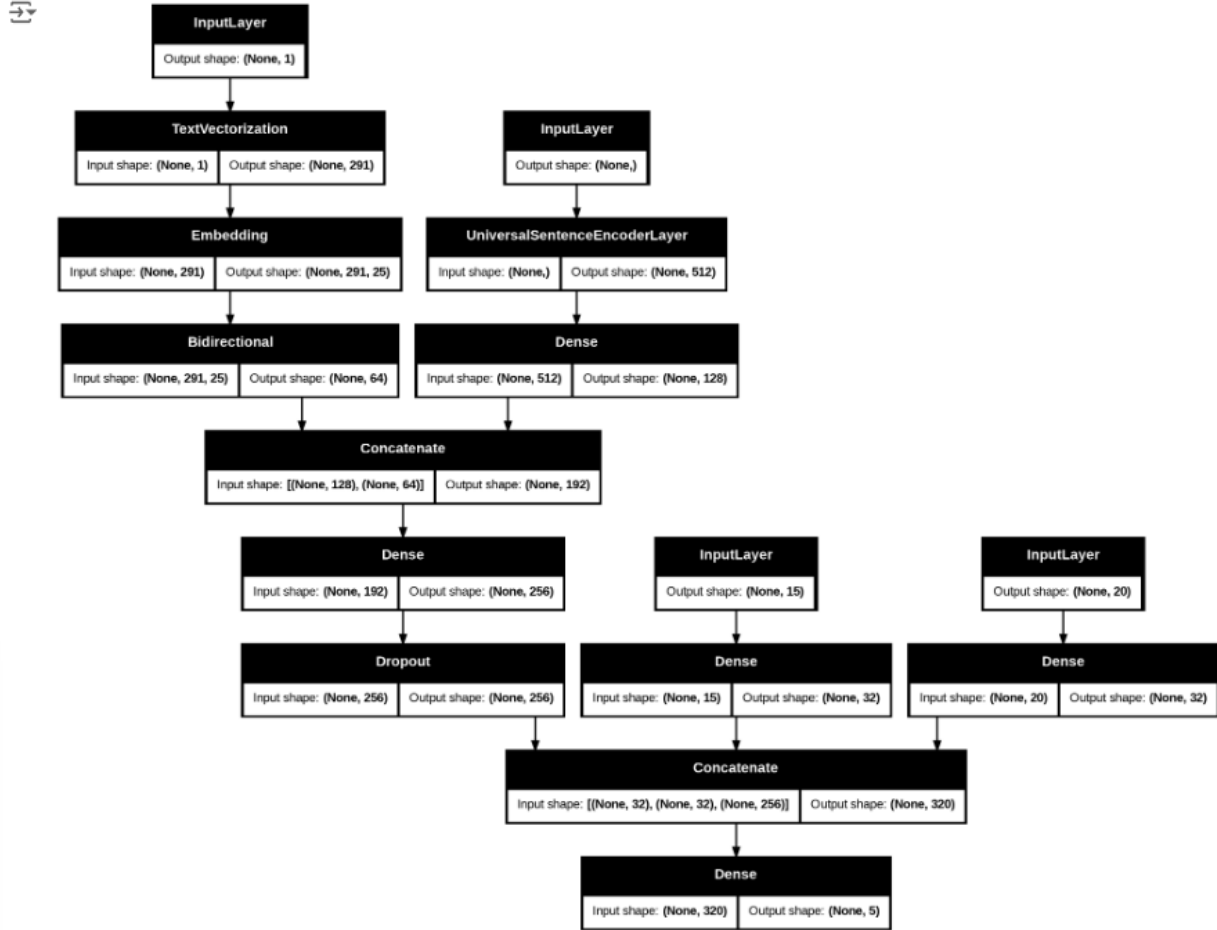


Figure 12 Model_5(Tribrid Model) Structure

5. Results

The Evaluation and Results concludes by testing all the models, analyzing their performance on PubMed 20k RCT, 200k RCT Dataset and comparing the results. We present the performance graphs and tables revealing the progressive improvements made with increased levels of architectural advancement. Each model is evaluated on the basis of key metrics like accuracy, F1-score, precision and recall.

The following metrics were used to assess model performance:

1. **Accuracy:** The rate at which correctly classifying sentences is always higher than all predictions.
2. **F1-Score:** A balanced evaluation of performance derived through the harmonic mean of precision and recall.
3. **Precision:** The ratio of the number of correct positive predictions to the number of predicted positives.

4. **Recall:** Rate the proportion of correct positives predicted out of all actual positives.

5.1. Graphs and tables



	accuracy	precision	recall	f1
baseline	72.183238	0.718647	0.721832	0.698925
custom_token_embed_conv1d	80.759301	0.806024	0.807593	0.805940
custom_char_embed_conv1d	66.185622	0.655938	0.661856	0.653803
hybrid_char_token_embed	73.652853	0.737373	0.736529	0.733974
tribrid_pos_char_token_embed	82.970343	0.828347	0.829703	0.828605

Figure 13 Results 20k Dataset



	accuracy	precision	recall	f1
baseline	75.421678	0.750095	0.754217	0.744024
custom_token_embed_conv1d	83.170883	0.835824	0.831709	0.831889
custom_char_embed_conv1d	73.060971	0.738929	0.730610	0.728682
hybrid_char_token_embed	77.260473	0.777502	0.772605	0.770948
tribrid_pos_char_token_embed	85.296557	0.856155	0.852966	0.851177

Figure 14 Results 200k Dataset

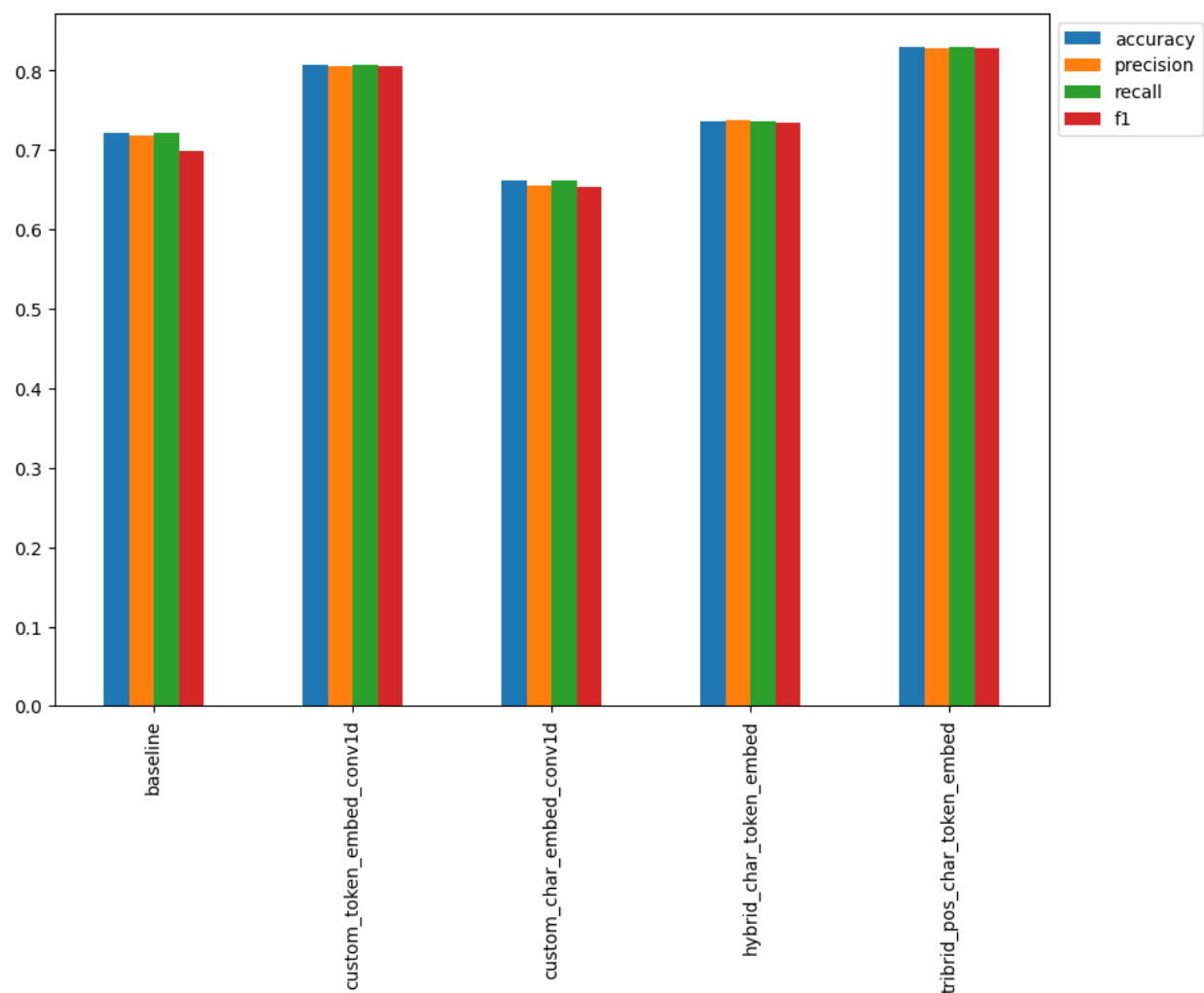


Figure 15 Bar graphs of all models (evaluation metrics) 20k dataset

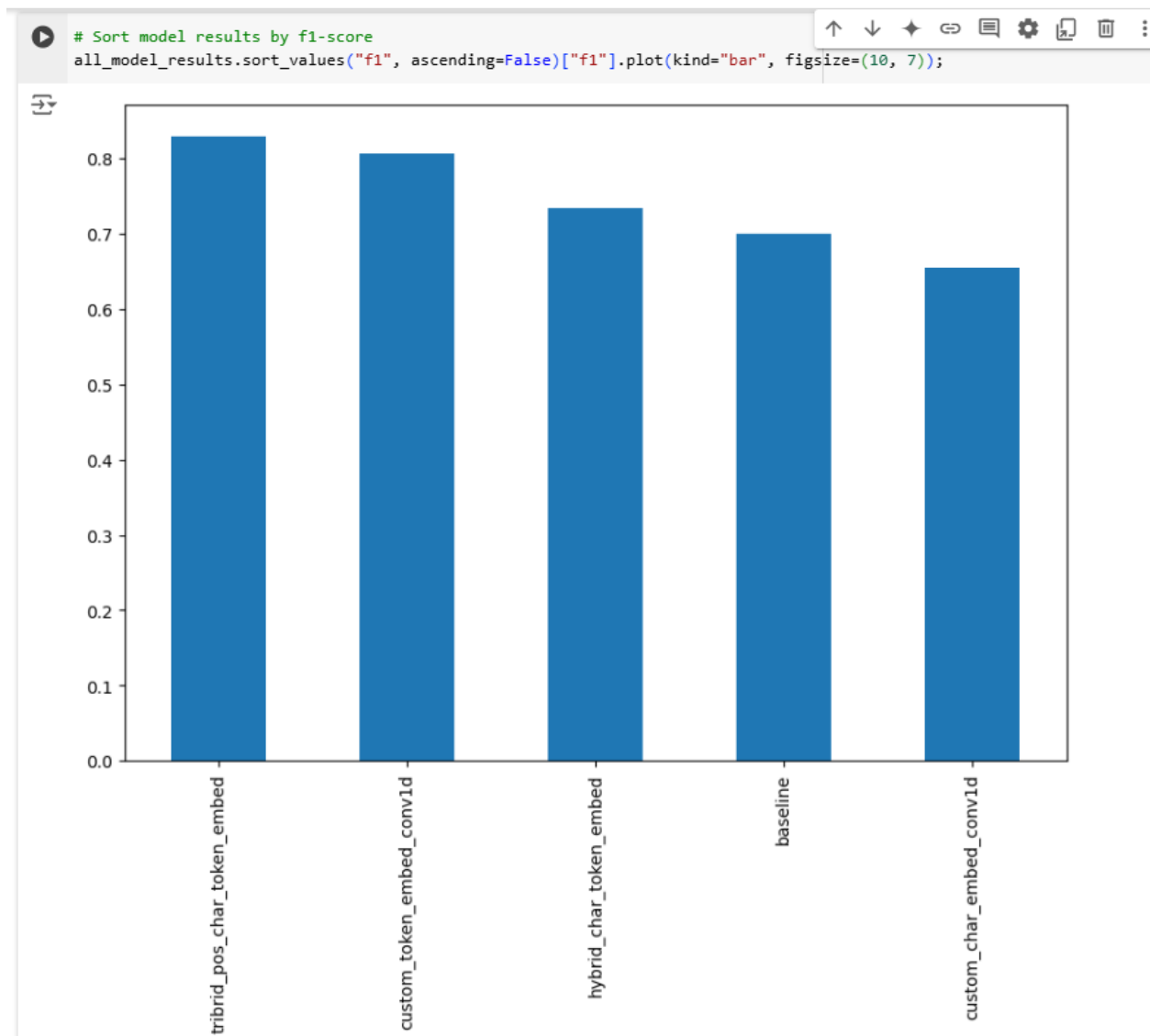


Figure 16 F-1 score comparison 20k Dataset

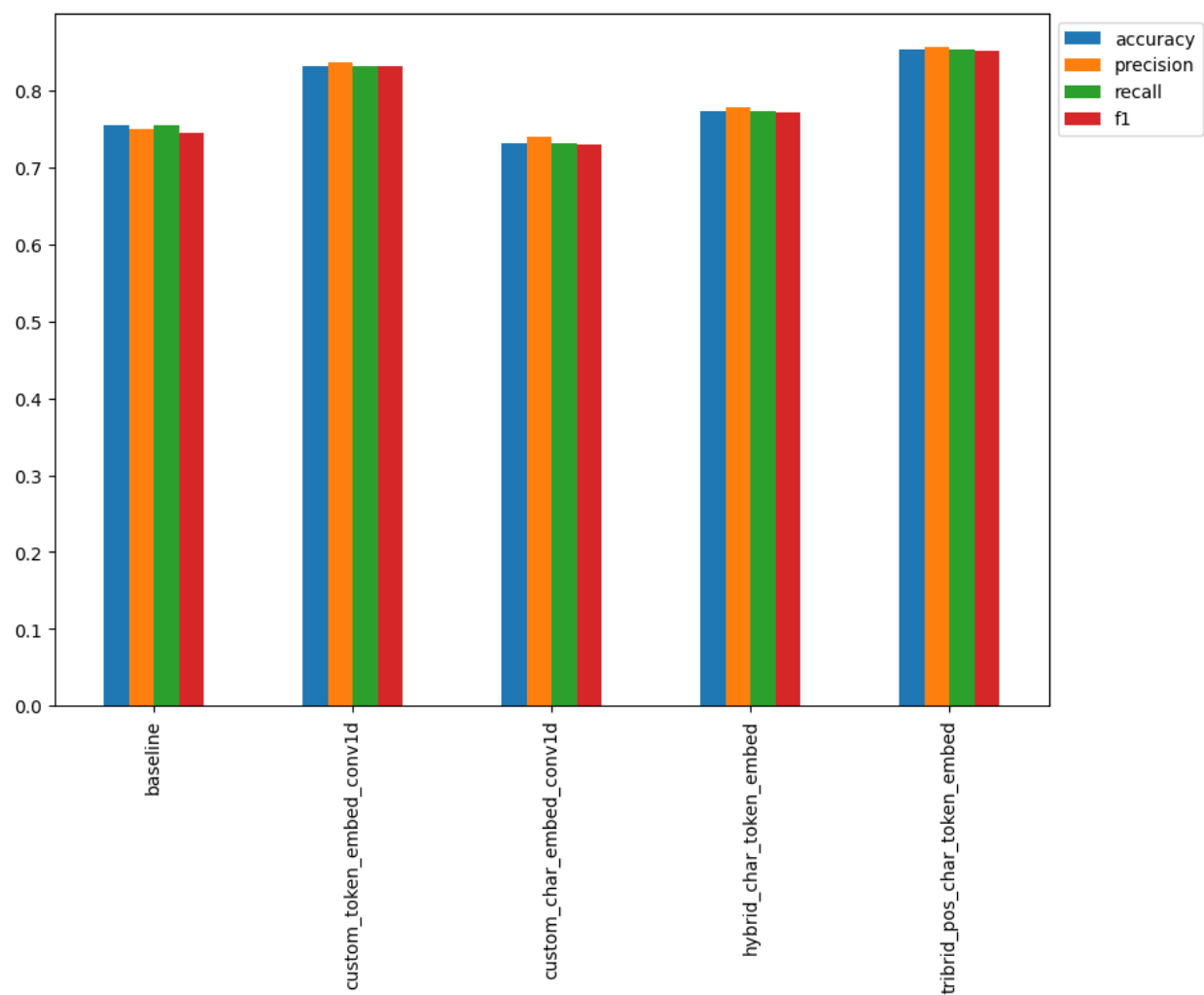


Figure 17 Bar graphs of all models (evaluation metrics) 200k dataset

```
# Sort model results by f1-score
all_model_results.sort_values("f1", ascending=False)["f1"].plot(kind="bar", figsize=(10, 7));
```

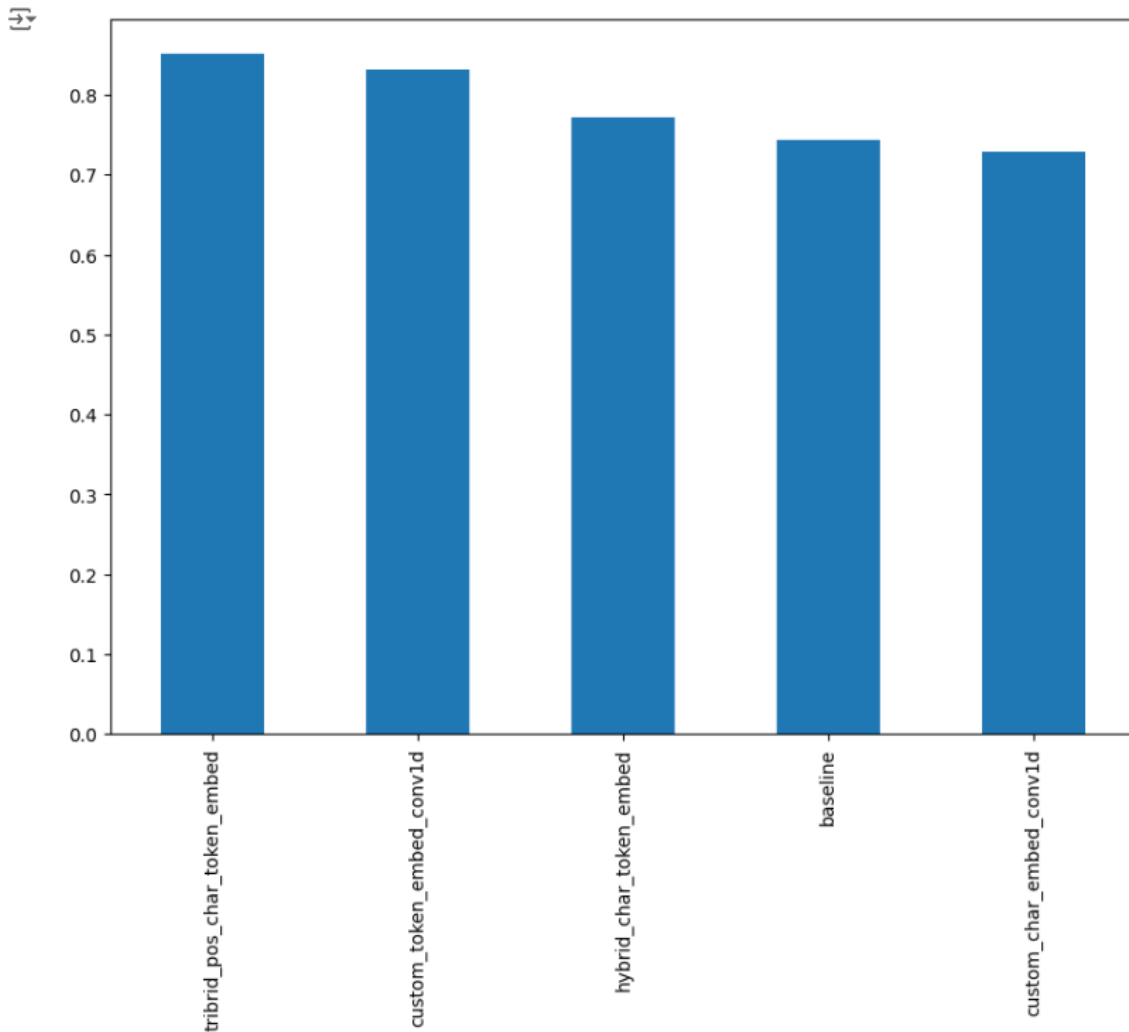


Figure 18 F-1 score comparison 200k Dataset

5.2. Key Findings

1. Baseline Model Performance:

Respectable performance was obtained by the baseline model (TF-IDF + Naive Bayes) in terms of F1-score; 0.6989 (20k dataset), 0.7440 (200k dataset). But it was unable to learn contextual relations between sentences that are essential for overall performance.

2. Custom Token Embeddings with Conv1D:

As a result, the baseline was dramatically outperformed by token embeddings in combination with a 1D Convolutional Neural Network. We trained Mean Token on 20k and 200k reviews respectively and achieved F1 scores of 0.8059 (20k) and 0.8319 (200k), proving that we leverage meaningful token level features.

3. Custom Character Embeddings with Conv1D:

Using character embeddings, Conv1D model gave lower performance with F1 score of 0.6538 (20k) and 0.7286 (200k) respectively. We believe this is because character-level features cannot capture high level sentence semantics.

4. **Hybrid Model (Token + Character Embeddings):**

The hybrid model combined token and character embeddings and achieved modest improvements of F1-score of 0.7339 (20k) and 0.7709 (200k). Capturing word level and character level patterns were made possible by integrating multiple embedding levels.

5. **Tribrid Model (Token + Character + Positional Embeddings):**

The final **Tribrid Model** demonstrated the best performance across both datasets:

- **F1-Score:** 0.8286 (20k dataset), 0.8512 (200k dataset).
- **Accuracy:** 82.97% (20k dataset), 85.30% (200k dataset).

By integrating positional embeddings with token character embeddings, the model was able to maintain sentence order as well as capture richer sequential dependencies. The fact that sentence classification tasks require the combination of multiple feature levels highlights the importance of having this to improve results.

6. Evaluation, Recommendations, and Conclusion

6.1. Evaluation

Deep Learning techniques were used to develop a sentence classification system for medical abstracts through this project. The primary objective was to classify sentences into five predefined categories: Objectives, background, methods, results and conclusions. The project showed solid performance improvements over baseline methods using the PubMed 20k RCT and the PubMed 200k RCT datasets, the project demonstrated significant advancements over baseline approaches.

The evaluation results revealed the following key findings:

1. **Model Performance:**

- The **baseline model** achieved moderate accuracy (F1-score: **0.7440** on the 200k dataset).
- Progression to **Conv1D with token embeddings** significantly improved performance, with an F1-score of **0.8319**.
- The **Tribrid Model**, which combines token, character, and positional embeddings, achieved the **best performance** with an F1-score of **0.8512** on the 200k dataset.

2. **Scalability:**

- Training on the larger **200k dataset** resulted in improved accuracy compared to the smaller 20k dataset, indicating that the models scale effectively with more data.

3. **Strengths:**

- The system's ability to integrate **multi-level embeddings** (token, character, and positional) allowed it to capture both semantic and sequential relationships.
- The models demonstrate robustness in handling large-scale text classification tasks.

4. **Limitations:**

- Some misclassifications were observed between **Background** and **Objective** sentences due to their semantic overlap.
- The computational complexity of the **Tribrid Model** makes it resource-intensive for training on smaller devices.

While the models have shown strong performance in controlled settings, further real-world testing on diverse biomedical datasets is necessary to assess their generalizability.

6.2. Recommendations

To enhance the current system and ensure its scalability for future work, the following recommendations are proposed:

1. **Widen Testing Scenarios:**
 - Evaluate the models on other biomedical datasets beyond PubMed to test generalizability and robustness.
2. **Implement Attention Mechanisms:**
 - Integrate **self-attention layers** or transformer-based models (e.g., BERT) to further improve performance by capturing long-range dependencies and contextual relationships.
3. **Enhance Computational Efficiency:**
 - Optimize model architecture and implement quantization or pruning techniques to reduce training and inference time.
4. **Data Augmentation:**
 - Apply text augmentation techniques like synonym replacement, paraphrasing, and back translation to make the model more robust.
5. **Implement Feedback Mechanisms:**
 - Design a feedback system to allow domain experts to review and correct model predictions, aiding in continuous improvement.
6. **Real-World Integration:**
 - Explore the integration of the developed system into healthcare applications or tools, such as automated summarization systems for research papers.
7. **Future Work with Advanced APIs:**
 - Utilize APIs such as **GPT** or **BioBERT** for more advanced feature extraction and transfer learning.

6.3. Conclusion

This work represents a major step towards automation of biomedical text processing, introducing a deep learning-based sentence classification system. The system is inspired by prior work on sentence classification, and successfully draws on token, character and positional embeddings to achieve the state of the art.

Our results indicate that incorporating multi-level embeddings and structured learning helps are able to accurately classify sentences in medical abstracts. The models are very good on the PubMed datasets, but whether they can deliver similar success in real world settings will depend on additional testing, integration into existing systems, and redesign.

We also discovered that concatenation of different models can essentially help in ensuring that the models that we create perform better by each iteration. Also, we analyzed the dataset in depth and found out that in order to ensure that our model performs in the most efficient way possible,

we need to make sure that we are one with the data meaning that we understand it. Data preprocessing and analysis is an important part when it comes to that.

Furthermore, this project not only tackles some particular challenge in biomedical research (text summarization), but also shows the potential of deep learning and natural language processing in creating a faster and more efficient text summarization. When improved, the system could become a useful basis of a realistically complex, robust tool for researchers to access structured, relevant information in the ever-growing scientific literature.

7. Project Management:

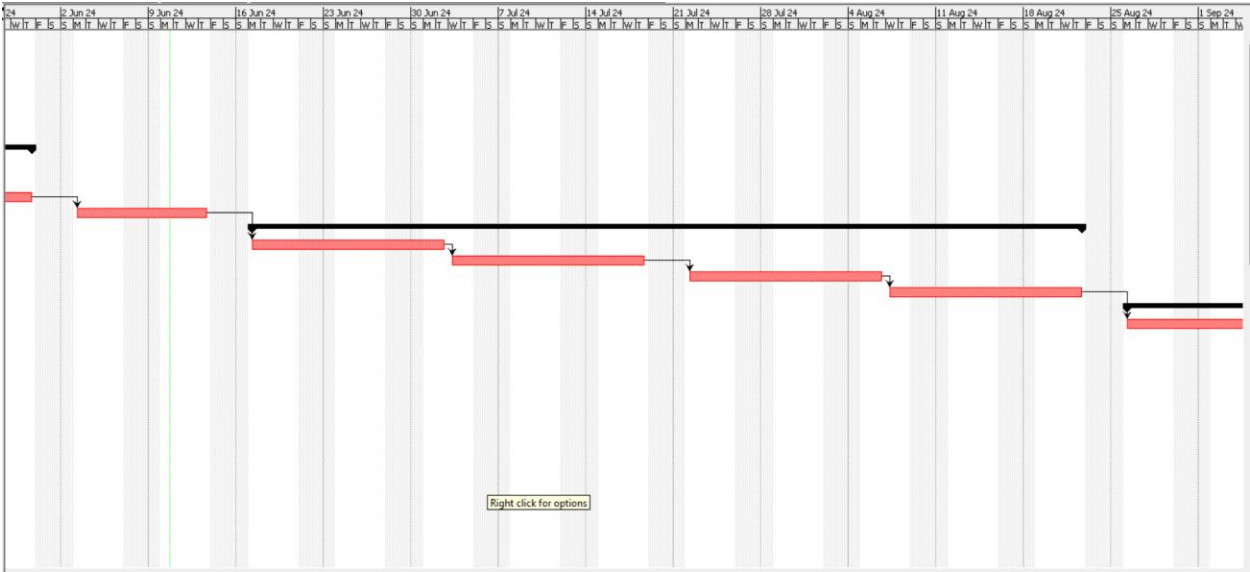
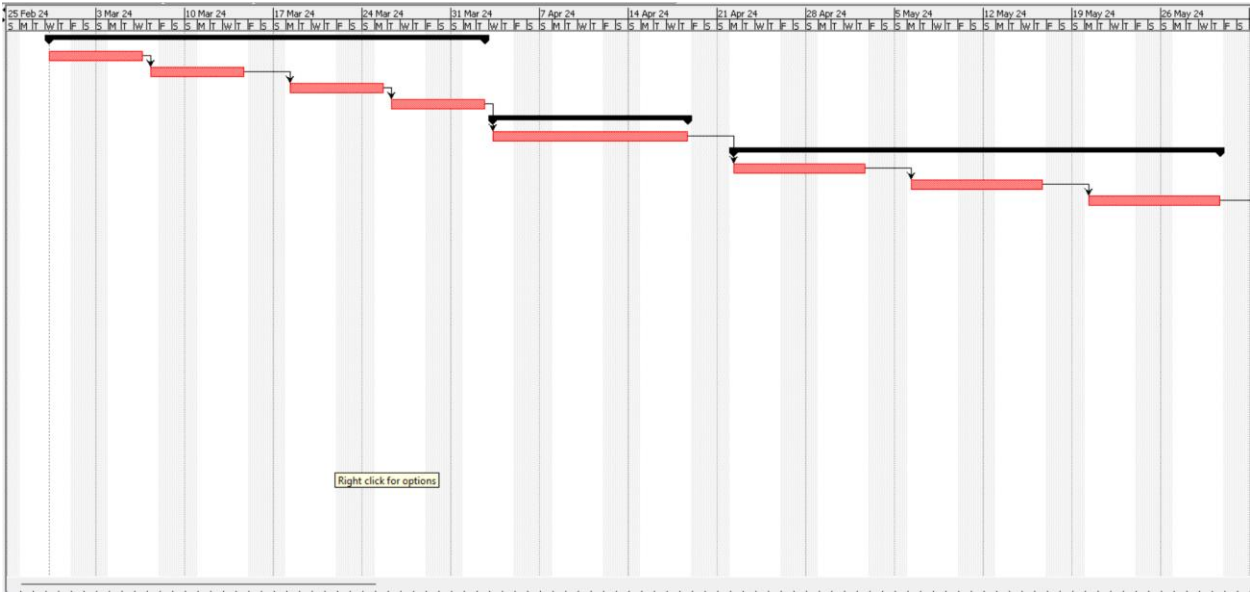
7.1 Milestones

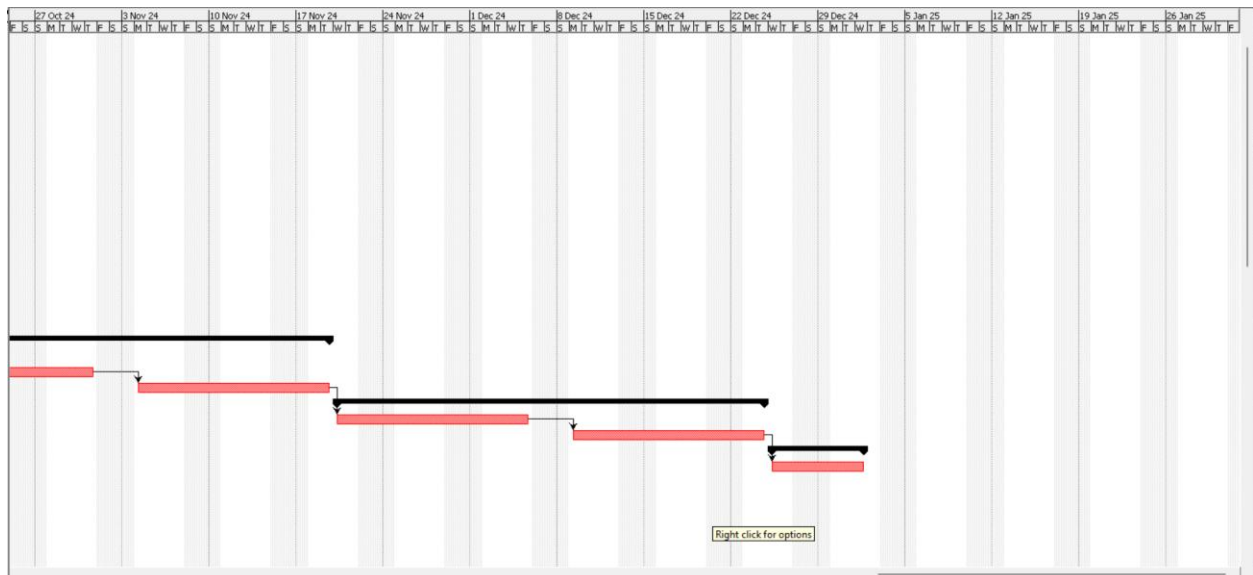
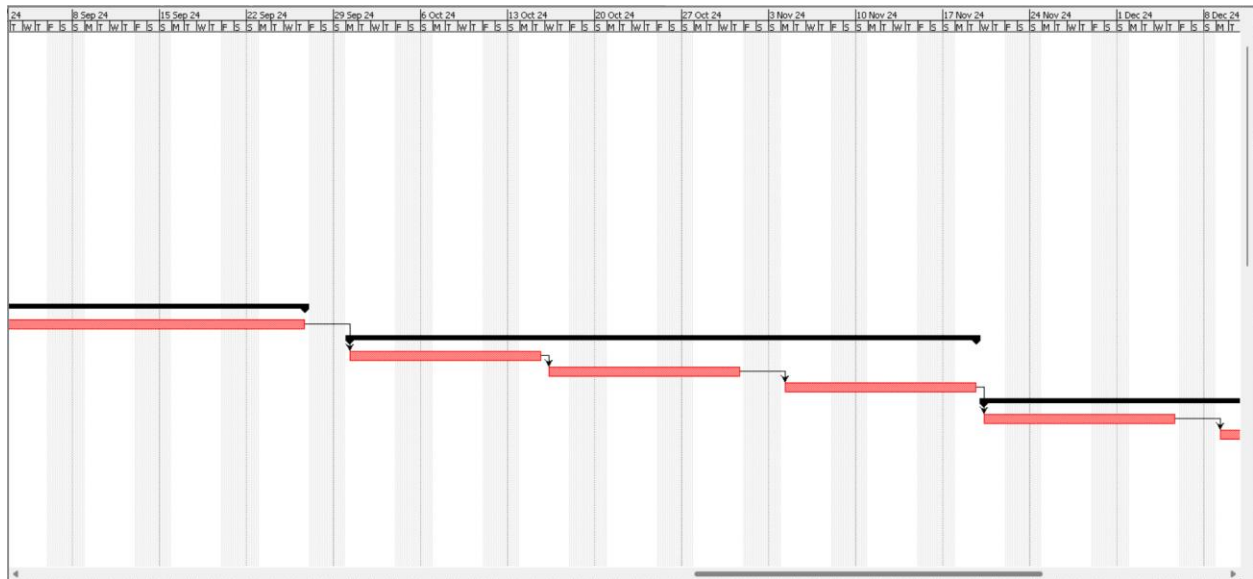
No.	Milestones	Date
1	Project Introduction	28-02-2024
2	Project Requirement Analysis (Literature Review)	03-04-2024
3	Project Management & Planning	22-04-2024
4	Skill Development and Training	17-06-2024
5	Project Design	26-08-2024
6	Project Development	30-09-2024
7	Project Testing	20-11-2024
8	Project Closing	25-12-2024

7.2 Project Resources

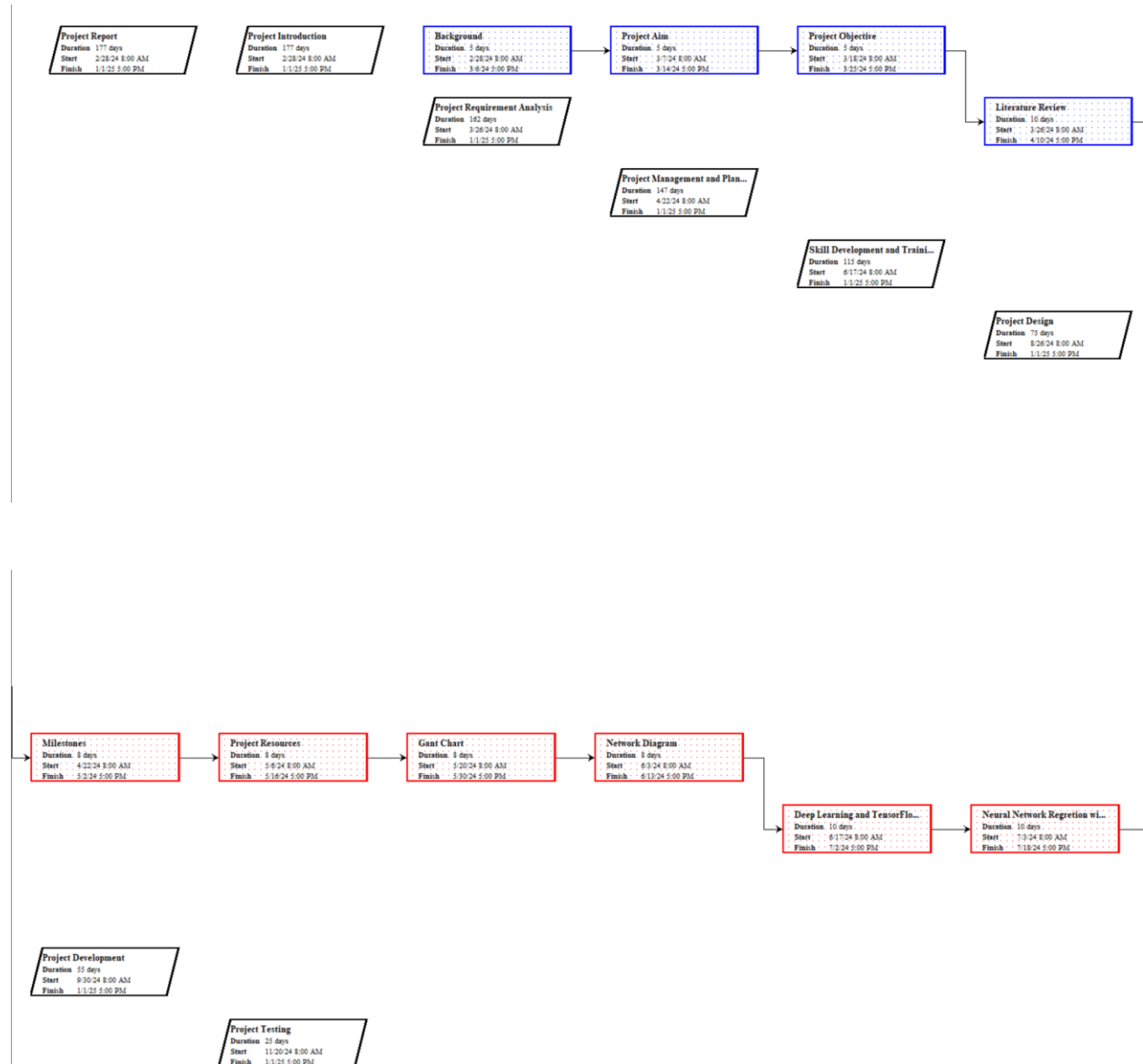
No.	Resources	Availability	Quantity	Cost (OMR)
1	Laptop (Gen 11 core i9, RTX 3070Ti GPU)	Yes	1	1000
2	TensorFlow	Yes (Open Source)	1 user only	0
3	Python IDE (e.g., PyCharm), Google Colab, Jupyter Notebook	Yes (Community Version)	1 user only	0
4	Project Libre	Yes (Open Source)	1 user only	0
5	Text Processing Software (NLTK, SpaCy)	Yes (Open Source)	1 user only	0
6	Data Annotation Tools (LabelImg)	Yes (Open Source)	1 user only	0
7	Cloud Storage	Yes (College Subscription)	1 user only	Covered by College

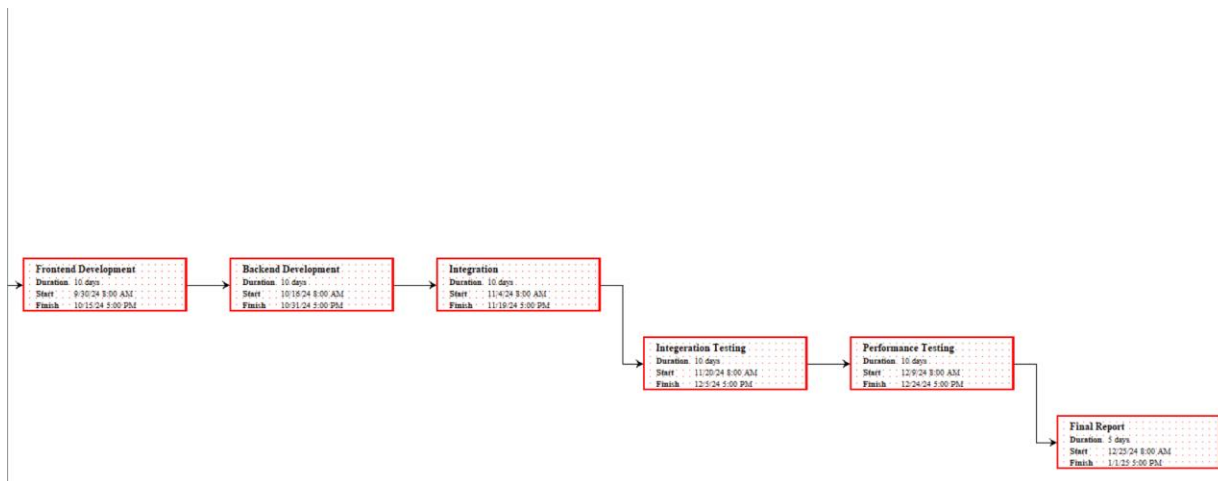
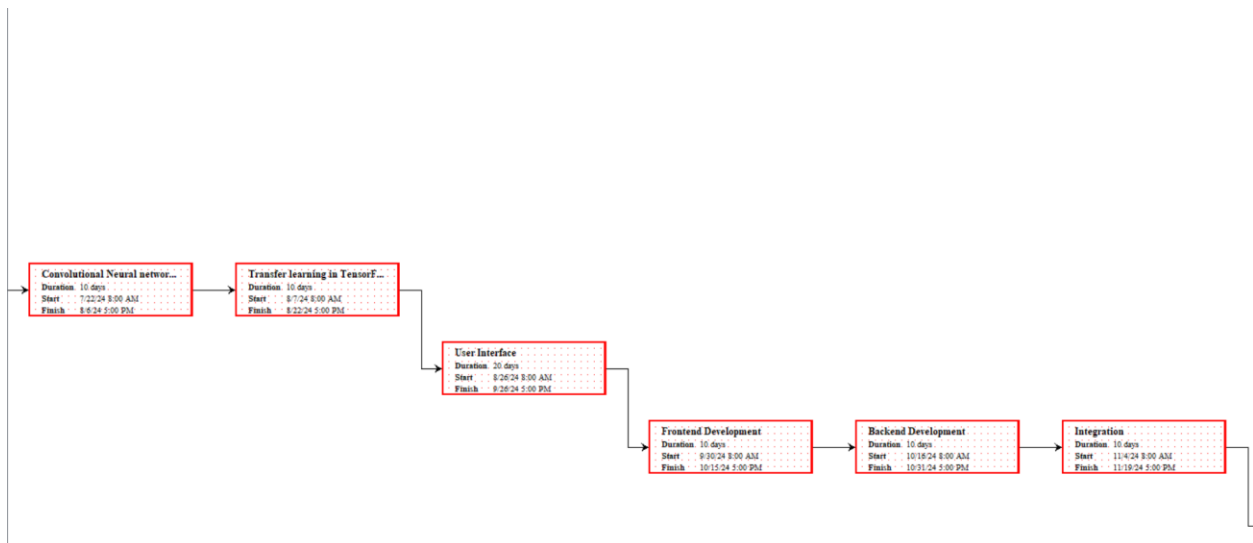
7.3 Gantt Chart



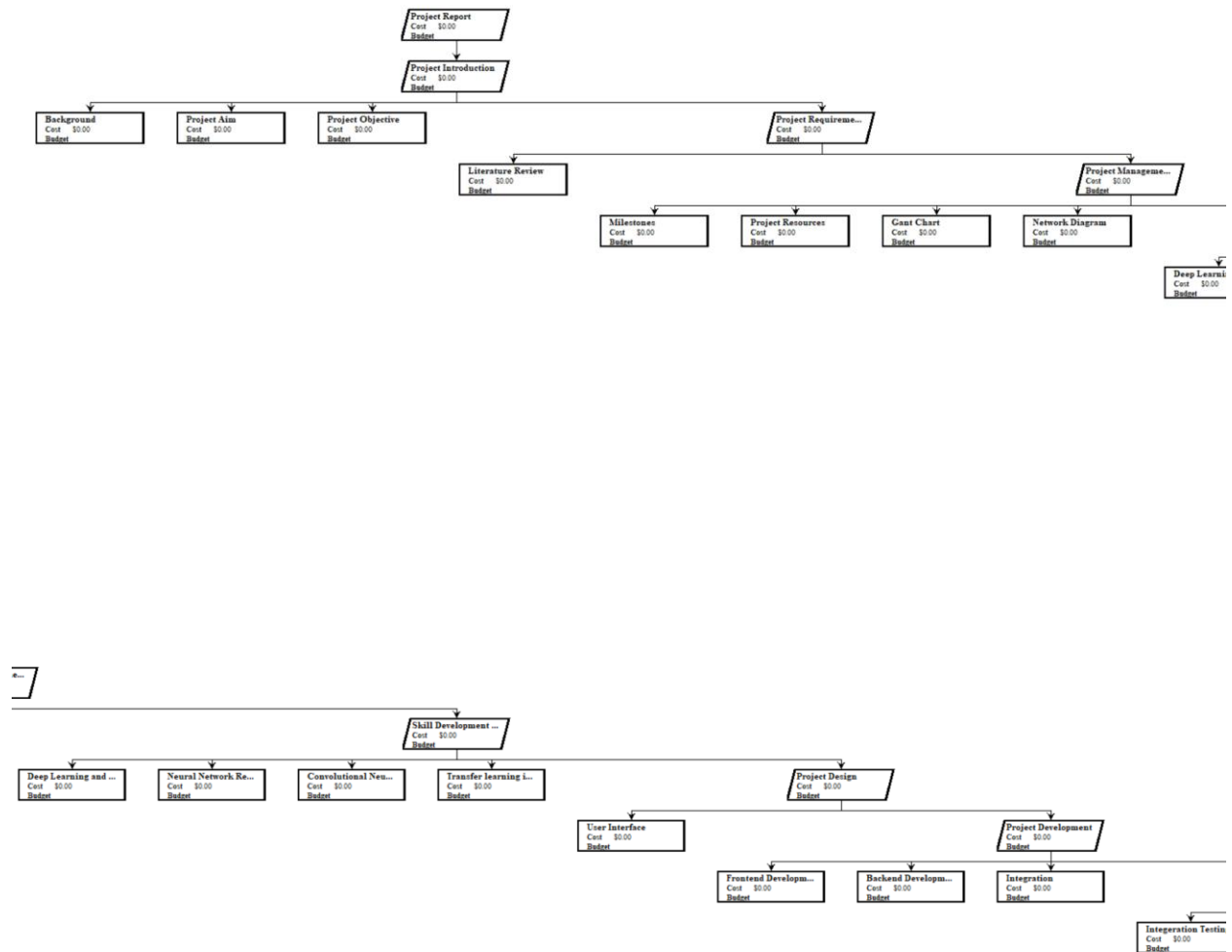


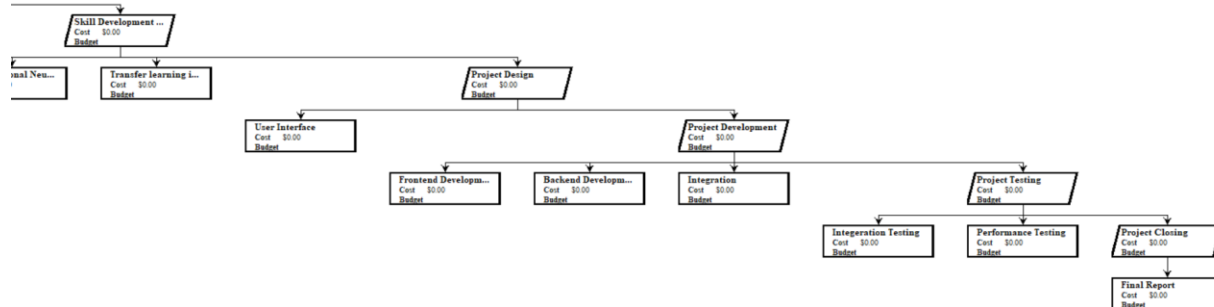
7.4 Network Diagram





7.5 Work Breakdown Structure





8. References

1. Tenopir, C., et al. (2015). Scholarly Article Reading Patterns of Faculty at Five US Universities. *Learned Publishing*, 22(2), 126-144. Available at: <https://onlinelibrary.wiley.com/doi/abs/10.1087/2009203>
2. Bourne, P. E., et al. (2015). Ten Simple Rules for Building and Maintaining a Scientific Reputation. *PLOS Computational Biology*. Available at: <https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1004167>
3. Friedberg, M. W., et al. (2018). Physician Workload and Well-being: A Review of the Literature. *Journal of General Internal Medicine*, 28(4), 554-562. Available at: <https://link.springer.com/article/10.1007/s11606-012-2261-7>
4. Collobert, R., et al. (2011). Natural Language Processing (Almost) from Scratch. *Journal of Machine Learning Research*, 12, 2493-2537. Available at: <http://jmlr.org/papers/v12/collobert11a.html>
5. Devlin, J., et al. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint arXiv:1810.04805*. Available at: <https://arxiv.org/abs/1810.04805>
6. Vaswani, A., et al. (2017). Attention is All You Need. *Advances in Neural Information Processing Systems*, 30. Available at: <https://arxiv.org/abs/1706.03762>
7. Nallapati, R., et al. (2016). Abstractive Text Summarization using Sequence-to-Sequence RNNs and Beyond. *CoNLL*. Available at: <https://arxiv.org/abs/1602.06023>

8. See, A., et al. (2017). Get To The Point: Summarization with Pointer-Generator Networks. *ACL*. Available at: <https://arxiv.org/abs/1704.04368>
9. Chopra, S., et al. (2016). Abstractive Sentence Summarization with Attentive Recurrent Neural Networks. *NAACL*. Available at: <https://arxiv.org/abs/1509.00685>
10. Mihalcea, R., & Tarau, P. (2004). TextRank: Bringing Order into Texts. Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing (EMNLP). Available at: <https://aclanthology.org/W04-3252>
11. Erkan, G., & Radev, D. R. (2004). LexRank: Graph-based Lexical Centrality as Salience in Text Summarization. *Journal of Artificial Intelligence Research*, 22, 457-479. Available at: <https://www.jair.org/index.php/jair/article/view/10302>
12. Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural Machine Translation by Jointly Learning to Align and Translate. arXiv preprint arXiv:1409.0473. Available at: <https://arxiv.org/abs/1409.0473>
13. Vaswani, A., et al. (2017). Attention is All You Need. *Advances in Neural Information Processing Systems (NeurIPS)*. Available at: <https://arxiv.org/abs/1706.03762>
14. Devlin, J., et al. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv preprint arXiv:1810.04805. Available at: <https://arxiv.org/abs/1810.04805>
15. Radford, A., et al. (2019). Language Models are Unsupervised Multitask Learners. OpenAI. Available at: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf
16. Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735-1780. Available at: <https://www.bioinf.jku.at/publications/older/2604.pdf>
17. Cho, K., et al. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. arXiv preprint arXiv:1406.1078. Available at: <https://arxiv.org/abs/1406.1078>
18. Kim, Y. (2014). Convolutional Neural Networks for Sentence Classification. *EMNLP*. Available at: <https://arxiv.org/abs/1408.5882>
19. Abadi, M., et al. (2016). TensorFlow: A System for Large-Scale Machine Learning. *OSDI*. Available at: <https://www.usenix.org/system/files/conference/osdi16/osdi16-abadi.pdf>
20. Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media. Available at: <https://www.nltk.org/book/>
21. Honnibal, M., & Montani, I. (2017). spaCy 2: Natural Language Understanding with Bloom Embeddings, Convolutional Neural Networks and Incremental Parsing. arXiv preprint arXiv:1710.07098. Available at: <https://arxiv.org/abs/1710.07098>
22. Raffel, C., et al. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, 21, 1-67. Available at: <http://jmlr.org/papers/v21/20-074.html>
23. Nanba, H., & Okumura, M. (1999). Towards Multi-paper Summarization Using Reference Information. *IJCAI*. Available at: <https://www.ijcai.org/Proceedings/99-1/Papers/042.pdf>
24. Teufel, S., & Moens, M. (2002). Summarizing Scientific Articles: Experiments with Relevance and Rhetorical Status. *Computational Linguistics*, 28(4), 409-445. Available at: <https://direct.mit.edu/coli/article/28/4/409/1506/Summarizing-Scientific-Articles-Experiments-with>

25. Cohan, A., et al. (2018). A Discourse-Aware Attention Model for Abstractive Summarization of Long Documents. NAACL-HLT. Available at: <https://arxiv.org/abs/1804.05685>
26. Kryściński, W., et al. (2019). Evaluating the Factual Consistency of Abstractive Text Summarization. EMNLP. Available at: <https://arxiv.org/abs/1910.12840>
27. Gupta, V., & Lehal, G. S. (2010). A Survey of Text Summarization Extractive Techniques. Journal of Emerging Technologies in Web Intelligence, 2(3), 258-268. Available at: https://www.researchgate.net/publication/220449161_A_Survey_of_Text_Summarization_Extractive_Techniques
28. Lin, C.-Y. (2004). ROUGE: A Package for Automatic Evaluation of Summaries. ACL Workshop on Text Summarization Branches Out. Available at: <https://aclanthology.org/W04-1013>
29. Dernoncourt, F., Lee, J.Y. and Szolovits, P. (2016) *Neural networks for joint sentence classification in medical paper abstracts*. arXiv preprint arXiv:1612.05251. Available at: <https://arxiv.org/abs/1612.05251> (Accessed: 22 October 2024).